COMSATS University Islamabad,

Sahiwal Campus.

Sign Bridge

(AI-Sign Language Translator)

A project presented to

COMSATS University Islamabad,

Sahiwal Campus.

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Zaryab Nadeem CIIT/FA22-BCS-124/SWL

Hafiz Muhammad Umair Rana CIIT/FA22-BCS-198/SWL

Muhammad Uzair CIIT/FA22-BCS-225/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Zaryab Nadeem	Hafiz Muhammad Umair Rana	Muhammad Uzair
---	---	---

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) "**Sign Bridge** (AI Sign Language Translator)" was developed by **Zaryab Nadeem** (CIIT/FA22-BCS-124), **Hafiz Muhammad Umair Rana** (CIIT/FA22-BCS-198) and **Muhammad Uzair** (CIIT/FA22-BCS-225) under the supervision of "**Dr. Yawar Abbas**" and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Sciences.

Supervisor

Dr. Yawar Abbas Abid

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Syed M Awais

Assistant Professor

Department of Computer Science

Air University Multan

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Effective communication remains one of the most significant challenges for the deaf and mute community, as traditional methods of interaction often rely heavily on spoken language, creating a substantial social and functional barrier between individuals with hearing or speech impairments and the general population. In many real-world scenarios, such individuals are required to depend on human interpreters or manual gesture-based communication, which may be slow, inconsistent, or not widely understood. These limitations highlight the urgent need for a practical, portable, and intelligent digital solution capable of enabling real-time and structured communication. This Final Year Project, Sign Bridge, is developed to address this problem by providing an AI-powered mobile communication and learning platform that supports seamless interaction between sign language users and the hearing community. The system allows users to perform hand gestures in front of a mobile camera and receive instant text-based output, with optional speech feedback, enabling efficient and natural communication in everyday situations.

The system is implemented as a cross-platform mobile application using the Flutter framework, ensuring consistent performance and user experience across different devices. Unlike traditional systems that rely on server-side processing, Sign Bridge performs all intelligent operations locally using on-device Artificial Intelligence, thereby eliminating the need for continuous internet connectivity and ensuring enhanced privacy and low-latency response. The core translation mechanism is based on a real-time gesture recognition pipeline in which the application captures live camera input and extracts 21 hand landmarks, each represented by three-dimensional coordinates (x, y, z), resulting in a 63-dimensional feature vector. These features undergo preprocessing and normalization, including wrist-relative transformation and scaling, to maintain consistency across different users and varying environmental conditions. The processed data is then passed to a TensorFlow Lite model, which performs fast and efficient gesture classification directly on the mobile device. To ensure smooth and stable performance, the inference process is controlled using a throttling mechanism of approximately 300 milliseconds, balancing responsiveness with computational efficiency.

The overall system follows a modular and feature-based architecture, where responsibilities are clearly separated into different layers to ensure maintainability and scalability. The application is structured into independent feature modules responsible for user interface and interaction logic, while reusable services handle core functionalities such as speech processing, text-to-speech conversion, and asset mapping. Application-wide state is managed using Provider, which enables efficient data flow and dynamic UI updates. In addition, a centralized theming system is

implemented to ensure visual consistency and improve user experience across all screens of the application.

The platform integrates multiple communication and learning functionalities, transforming it into a comprehensive accessibility solution rather than a simple translation tool. It supports real-time sign-to-text recognition, speech-to-text transcription for voice input, and text-to-speech output for audio feedback. Additionally, the system includes a text-to-sign module that visually represents text input using sign language assets, allowing users to understand gestures more effectively. To further enhance learning and engagement, the application provides an interactive sign language dictionary and a daily challenge module that presents dynamic tasks to help users practice and reinforce their understanding of sign language. These features collectively create a complete communication and learning ecosystem that benefits both deaf users and individuals interested in learning sign language.

The system is developed following a structured software engineering approach, including functional and non-functional requirement analysis, use case modeling, system design, implementation, and testing. It demonstrates the practical integration of Artificial Intelligence, Computer Vision, Mobile Application Development, and Human Computer Interaction principles in a real-world accessibility-focused application. Overall, this Final Year Project represents a meaningful and impactful solution that leverages modern technology to promote inclusivity and accessibility. By combining real-time AI processing, efficient mobile system design, and user-centered interaction, Sign Bridge provides a practical, scalable, and intelligent approach to reducing communication barriers faced by the deaf and mute community.

Acknowledgement

All praise and gratitude are due to Almighty Allah, the Most Beneficent and the Most Merciful, who blessed us with health, patience, strength, and the opportunity to acquire knowledge. It is through His infinite mercy and guidance that we were able to successfully complete this project. We acknowledge that every achievement and accomplishment is only possible through His countless blessings and wisdom.

We would like to express our deepest gratitude and sincere appreciation to our respected project supervisor, **Dr. Yawar Abbas Abid**, whose invaluable guidance, expert advice, and continuous encouragement played a vital role throughout the development of this project. Their insightful suggestions, constructive criticism, and unwavering support helped us overcome numerous challenges encountered during the course of this work. Their dedication to academic excellence and willingness to share knowledge greatly contributed to the successful completion of this project. Without their personal supervision and mentorship, this achievement would not have been possible.

We owe a special debt of gratitude to our beloved parents and family members. Their unconditional love, sacrifices, patience, and constant prayers have always been a source of strength and inspiration for us. They have instilled in us the values of honesty, hard work, perseverance, and integrity, which have guided us throughout our academic journey. Their unwavering support and belief in our abilities motivated us to strive for excellence and successfully accomplish this project.

We are also thankful to all our teachers and faculty members who have contributed to our academic and professional development throughout our degree program. Their knowledge, experience, and commitment to education have provided us with a strong foundation and inspired us to pursue learning with dedication and enthusiasm. We dedicate this achievement to all those who supported and encouraged us throughout this endeavor, and we remain sincerely grateful for their contributions and goodwill.

Zaryab Nadeem Hafiz Muhammad Umair Rana Muhammad Uzair

--- --- ---

Abbreviations

AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
MERN	MongoDB, Express.js, React, Node.js
HCI	Human Computer Interaction
API	Application Programming Interface
UC	Use Case
FR	Functional Requirement
NFR	Non-Functional Requirement
ID	Identifier
FYP	Final Year Project
UI	User Interface
TFLite	TensorFlow Lite
SVM	Support Vector Machines
UAT	User Acceptance Testing

Table of Contents

1 Introduction 1

1.1 Brief 1

1.2 Relevance to Course Modules 2

1.3 Project Background 4

1.4 Literature Review 5

1.5 Analysis from Literature Review 6

1.6 Methodology and Software Lifecycle for This Project 7

1.7 Rationale behind Selected Methodology 9

2 Problem Definition 12

2.1 Problem Statement 12

2.2 Deliverables and Development Requirements 14

2.3 Current System 16

3 Requirement Analysis 21

3.1 Use Case Diagram 21

3.2 Detailed Use Cases 23

3.2.1 Authenticate User (UC-1) 23

3.2.2 Recognize Sign Gesture (Sign → Text) 23

3.2.3 Translate Text to Speech 24

3.2.4 Sign Language Dictionary 25

3.2.5 Practice Sign & Accuracy Testing 26

3.2.6 Manage User Preferences 26

3.2.7 Manage User Profile 27

3.2.8 Daily Challenge 28

3.3 Functional Requirements 29

3.4 Non-Functional Requirements 32

3.4.1 Performance 32

3.4.2 Usability 32

3.4.3 Reliability 32

3.4.4 Accuracy 33

3.4.5 Security and Privacy 33

3.4.6 Maintainability 33
3.4.7 Portability 34
3.4.8 Scalability 34
4 Design and Architecture 37
4.1 System Architecture 37
4.1.1 Presentation Layer 37
4.1.2 Application Layer 38
4.1.3 Machine Learning Processing Layer 38
4.1.4 Data Layer 39
4.1.5 System Architecture Summary 39
4.1.6 System Architecture Diagram 41
4.2 Data Representation 42
4.2.1 User Entity 42
4.2.2 Gesture Record Entity 42
4.2.3 Sign Dictionary Entity 42
4.2.4 Preferences Entity 42
4.2.5 Data Representation Summary 42
4.3 Process Flow / Representation 44
4.3.1 Sign Recognition Process Flow 44
4.3.2 Authentication and Profile Management Flow 45
4.3.3 Practice Module Flow 45
4.4 Design Models 47
4.4.1 Sequence Diagram 47
4.4.2 Class Diagram 48
5 Implementation 50
5.1 System Implementation Overview 51
5.2 Gesture Recognition Implementation 53
5.3 Machine Learning Model Implementation 54
5.4 Mobile Application Implementation 56
5.4.1 User Authentication 57
5.4.2 Real-time Gesture Capture 57
5.4.3 Predicted Text Output 57

5.4.4 Sign Language Dictionary 58
5.5 Integration of System Components 58
5.6 User Interface Implementation 60
5.6.1 Home Screen 61
5.6.2 Login and Signup Screen 61
5.6.3 Gesture Detection Screen 63
5.6.4 Learning Screen 63
5.6.5 Practice Screen 64
5.6.6 Profile Screen 65
5.6.6 Human Computer Interaction (HCI) 66
5.7 Data Handling and Storage 66
6 Testing and Evaluation 69
6.1 Testing Strategy 70
6.1.1 Unit Testing 71
6.1.2 Integration Testing 72
6.1.3 Functional Testing 73
6.1.4 Non-Functional Testing 74
6.1.5 User Acceptance Testing (UAT) 76
6.2 Results Analysis and Discussion 76
6.3 Limitations of Testing 78
7 Conclusion and Future Work 81
7.1 Conclusion 82
7.2 Achievements vs Objectives 84
7.3 Limitations of the Current System 86
7.4 Future Work and Enhancements 88
7.5 Social Impact and Significance 91
References 93

List of Figures

Figure 1-1 SDLC ____ 9
Figure 3-1 Use Case Diagram ____ 22
Figure 4-1 System Architecture Diagram ____ 41
Figure 4-2 Entity Relationship Diagram ____ 43
Figure 4-3 System Process Flow Diagram ____ 46
Figure 4-4 Sequence Diagram ____ 47
Figure 4-5 Class Diagram ____ 48
Figure 5-1 Implementation Overview ____ 52
Figure 5-2 Layered System Architecture ____ 56
Figure 5-3 Home Screen ____ 61
Figure 5-4 Login Screen ____ 62
Figure 5-5 Signup Screen ____ 62
Figure 5-6 Gesture Detection Screen ____ 63
Figure 5-7 Learning Screen ____ 64
Figure 5-8 Practice Screen ____ 64
Figure 5-9 Profile Screen ____ 65
Figure 5-10 Profile Screen 2 ____ 65
Figure 5-11 HCI ____ 66
Figure 6-1 Testing Strategy ____ 71
Figure 7-1 Achievements and Outcomes ____ 84

List of Tables

Table 2.1 Deliverables of the Sign Language 15
Table 2.2 Development Requirements 16
Table 2.3 Comparison of Current Systems 18
Table 3.1 Use Case: Authenticate User 23
Table 3.2 Use Case: Recognize Sign Gesture 24
Table 3.3 Use Case: Translate Text to Audio 24
Table 3.4 Use Case: Sign Dictionary 25
Table 3.5 Use Case: Practice Sign & Accuracy 26
Table 3.6 Use Case: Manage User Preferences 26
Table 3.7 Use Case: Manage User Profile 27
Table 3.8 Use Case: Daily Challenge 29
Table 3.9 Functional Requirements 31
Table 3.10 Non-Functional Requirements 34
Table 4.1 System Architecture Layers 40
Table 4.2 Data Entities and Attributes 43
Table 6.1 Unit Test Cases 72
Table 6.2 Key Integration Test Cases 73
Table 6.3 Functional Test Results Summary 74
Table 6.4 Accuracy Evaluation 75
Table 7.1 Achievements vs Objectives 85

Chapter 1
Introduction

1 Introduction

The rapid advancement of mobile computing and artificial intelligence has transformed the way humans interact with technology in daily life. Despite these improvements, communication barriers between deaf and mute individuals and the hearing population still remain a major challenge in many societies. Sign language is the primary mode of communication for people with hearing or speech impairments, but a lack of understanding among the general public often leads to miscommunication and social exclusion. This gap highlights the need for an efficient, accessible, and real-time solution that can bridge communication between both communities.

To address this issue, modern technologies such as computer vision, machine learning, and mobile application development are being combined to create intelligent assistive systems. These systems aim to provide real-time translation between sign language, text, and speech using commonly available devices like smartphones. The Sign Bridge AI Sign Language Translator is one such solution that leverages these technologies to enable smooth and natural communication. By integrating gesture recognition and speech processing into a single mobile platform, the system aims to improve accessibility, inclusivity, and independence for individuals with hearing and speech impairments.

1.1 Brief

The **Sign Bridge** AI Sign Language Translator is an AI-powered, mobile-based communication and learning application developed to reduce the communication gap between deaf, mute, and hearing individuals. The system is designed to provide an accessible, practical, and real-time solution that enables translation between sign language, text, and speech in everyday communication scenarios. The primary goal of the application is to facilitate smooth and natural interaction without requiring human interpreters or specialized hardware devices.

Traditional communication between deaf individuals and the general population often relies on interpreters, written notes, or typing-based interaction. These approaches are often slow, inconvenient, or not always available in real-world situations. The proposed system addresses this limitation by leveraging modern computer vision and on-device artificial intelligence techniques to recognize sign gestures using a smartphone camera and convert them into meaningful outputs.

The system is implemented as a cross-platform mobile application using the Flutter framework, ensuring compatibility and consistent performance across different devices. The gesture recognition pipeline is based on extracting 21 hand landmarks, where each landmark contains three-dimensional coordinates (x, y, z), resulting in a 63-dimensional feature vector. These features are preprocessed using a normalization technique that includes wrist-relative transformation and scaling, making the system robust to variations in hand position and size.

The processed feature vector is then passed to a **TensorFlow Lite (TFLite)** [1] model, which performs real-time gesture classification directly on the device. This on-device inference approach ensures low latency, efficient performance, and enhanced user privacy, as no data is transmitted to external servers. The prediction process is optimized using a throttled inference mechanism (~300 milliseconds interval) to maintain a balance between responsiveness and computational stability.

In addition to gesture recognition, the system integrates multiple communication and learning modules, including:

- Sign-to-Text for real-time gesture translation
- Speech-to-Text for live voice transcription
- Text-to-Speech for audio output
- Text-to-Sign using visual sign assets
- Interactive Sign Dictionary for learning support
- Daily Challenge module for user engagement and practice

The application follows a feature-based modular architecture with Provider state management, ensuring clean separation of responsibilities, scalability, and maintainability. Each module operates independently while contributing to a unified user experience.

The system demonstrates the feasibility of real-time, mobile-based sign language translation using lightweight AI models and efficient processing techniques. It serves not only as a communication tool but also as a learning platform, contributing toward improved accessibility, inclusivity, and independence for individuals with hearing and speech impairments.

1.2 Relevance to Course Modules

The development of the Sign Bridge AI Sign Language Translator is directly related to multiple core courses studied during the Bachelor of Science in Computer Science program. The project

represents a comprehensive integration of theoretical knowledge and practical skills into a real-world, accessibility-focused application. It demonstrates how different domains of computer science can be combined to build a meaningful and socially impactful system.

The project is strongly aligned with Software Engineering principles. It involves systematic requirement gathering, use case identification, architectural design, implementation planning, testing strategies, and structured documentation. The development process follows a well-defined Software Development Life Cycle (SDLC) with an Agile-based iterative approach. The application is designed using a feature-based modular architecture, where different functionalities are separated into independent modules. This approach improves maintainability, scalability, and code organization. Additionally, the use of Provider state management reflects modern software design practices for managing application state efficiently.

The system also has a strong connection with Artificial Intelligence and Machine Learning. The gesture recognition component is formulated as a classification problem, where hand landmark data is used as input to predict corresponding sign labels. Instead of using heavy deep learning models, the project utilizes a lightweight Tensor Flow Lite (TFLite) model for on-device inference. This demonstrates practical understanding of concepts such as feature extraction, data preprocessing, normalization, model optimization, and real-time prediction. The ability to deploy machine learning models directly on mobile devices highlights knowledge of edge AI and efficient model deployment techniques.

The project incorporates key concepts from Computer Vision through the use of hand landmark detection techniques. The system processes real-time camera input to extract 21 hand landmarks, each represented by three-dimensional coordinates. Understanding coordinate systems, spatial representation, feature transformation, and real-time frame processing is essential for implementing this functionality. The landmark-based approach also demonstrates how visual data can be converted into structured numerical input for machine learning models.

The development of the mobile application using the Flutter framework establishes a strong connection with Mobile Application Development and Web and Mobile Engineering courses. The project implements modern mobile development practices, including responsive UI design, asynchronous programming, real-time data handling, and efficient rendering of dynamic content. The application structure follows a clean separation between UI, services, and state management layers, reflecting professional mobile development standards.

The system also reflects principles from Human Computer Interaction (HCI). Since the primary users include individuals with hearing and speech impairments, the interface is designed with a strong focus on accessibility and usability. The application ensures a simple and intuitive layout, clear visual feedback, readable text output, and minimal navigation complexity. Features such as real-time gesture display, speech output, and interactive learning modules enhance user engagement and improve overall usability.

1.3 Project Background

Communication barriers faced by deaf and mute individuals represent a significant social, technological, and accessibility-related challenge in modern society. In countries such as Pakistan, as well as in many other parts of the world, sign language serves as the primary means of communication for individuals with hearing or speech impairments. However, a large portion of the general population lacks familiarity with sign language, which creates a considerable gap in communication. This gap becomes especially evident in critical environments such as educational institutions, healthcare facilities, workplaces, and public service areas, where effective communication is essential for participation, safety, and inclusion.

The inability of the general population to understand sign language often leads to social isolation, reduced opportunities, and limited access to essential services for individuals with disabilities. In many real-world scenarios, such as doctor–patient interactions, classroom discussions, or customer service communication, the absence of a reliable translation mechanism can significantly hinder effective interaction. This highlights the urgent need for practical and scalable solutions that can facilitate seamless communication between sign language users and non-signers.

Traditional approaches to addressing this issue, such as hiring professional interpreters, are not always feasible due to their limited availability, high cost, and dependency on scheduling. In many regions, especially in developing countries, access to trained interpreters is extremely restricted. Written communication is often used as an alternative; however, it is inherently slower and less efficient, particularly in real-time conversations where immediate feedback is required. Additionally, not all users are comfortable or proficient in written language, further limiting its effectiveness as a communication medium.

With the rapid advancement of technology, particularly in the fields of artificial intelligence, mobile computing, and computer vision, new opportunities have emerged to address these challenges. Modern frameworks such as Media Pipe have enabled real-time hand tracking and landmark detection using standard device cameras. These frameworks eliminate the need for specialized hardware such as sensor-based gloves or depth cameras, which were previously required for gesture recognition systems. As a result, it has become possible to develop lightweight, cost-effective, and mobile-friendly applications capable of performing gesture recognition in real time.

The core idea behind the Sign Bridge project originates from the need to design a practical, accessible, and deployable solution that can operate efficiently on everyday mobile devices. Instead of relying on computationally expensive deep learning techniques, such as convolutional neural networks (CNNs) applied directly to raw images, the system adopts a more efficient approach based on structured feature extraction. Hand gestures are first processed using landmark detection techniques, where 21 key points of the hand are identified and represented as numerical coordinates. These landmarks are then transformed into a structured feature vector that captures the spatial configuration of the gesture.

This feature-based representation significantly reduces computational complexity while preserving the essential information required for gesture classification. The extracted features are then processed using supervised machine learning algorithms or optimized on-device models such as TensorFlow Lite classifiers. This approach ensures that the system remains lightweight, efficient, and capable of real-time performance on mobile platforms without requiring high-end processing resources.

By combining mobile application development using Flutter, real-time computer vision techniques, and efficient machine learning models, the project presents a scalable and accessible solution for assistive communication. The integration of these technologies into a single platform ensures that the system can operate seamlessly in real-world environments while maintaining usability and performance.

1.4 Literature Review

Over the past decade, significant research has been conducted in the field of sign language recognition (SLR), with various approaches proposed to improve accuracy, efficiency, and usability. Researchers have explored multiple techniques, including image-based deep learning

models, sensor-based systems, and landmark-based classification methods, each with its own advantages and limitations.

One of the most widely studied approaches involves the use of Convolutional Neural Networks (CNNs) for classifying sign gestures directly from image data. These models are capable of automatically extracting spatial features and patterns from images, which allows them to achieve high accuracy, particularly in controlled datasets. However, CNN-based methods are often computationally expensive and require substantial processing power, such as GPU support. This makes them less suitable for deployment on mobile devices, where computational resources and energy consumption are limited.

Another category of research focuses on sensor-based systems, which utilize devices such as data gloves or motion sensors to capture hand movements. These systems can provide highly accurate gesture recognition because they directly measure hand position and movement. Despite their accuracy, they rely on additional hardware, which increases cost and reduces practicality for everyday use. As a result, such systems are not widely adopted in real-world scenarios, especially in low-resource environments.

More recent research trends have shifted towards the use of hand landmark detection frameworks such as Media Pipe. These frameworks enable the extraction of key hand points using standard cameras, eliminating the need for specialized hardware. The extracted landmarks are then used as input features for traditional machine learning algorithms, including Support Vector Machines (SVM), Random Forest, K-Nearest Neighbors (KNN), and Multi-Layer Perceptron (MLP). This approach significantly reduces computational complexity while still maintaining reasonable classification accuracy, making it more suitable for real-time and mobile-based applications.

Overall, the literature suggests that while image-based and sensor-based approaches are effective in controlled environments, there is still a strong need for practical, lightweight, and mobile-friendly solutions. Systems that combine efficient feature extraction with optimized machine learning models offer a promising direction for developing real-world sign language translation applications.

1.5 Analysis from Literature Review

The analysis of existing research and related systems highlights several important observations regarding the design and implementation of sign language recognition solutions. First, deep

learning models such as Convolutional Neural Networks (CNNs) demonstrate strong performance and high accuracy in gesture classification tasks. However, these models typically require significant computational resources, including GPU support, which limits their suitability for real-time mobile deployment. Second, sensor-based approaches provide high precision by capturing detailed hand movement data, but they depend on additional hardware such as data gloves or motion sensors, which reduces portability and increases cost. Third, landmark-based machine learning approaches offer a balanced trade-off between computational efficiency and classification accuracy, making them more practical for real-world applications.

The proposed Sign Bridge system aligns with the third category by adopting a landmark-based recognition approach. Instead of focusing solely on maximizing accuracy through complex neural architectures, the system prioritizes real-time usability, computational efficiency, and accessibility. By utilizing hand landmark detection through Media Pipe and converting these landmarks into 63-dimensional feature vectors, the system significantly reduces processing overhead compared to raw image-based methods. This approach allows efficient execution on mobile devices while maintaining acceptable performance.

Unlike many research prototypes that focus only on model development, the proposed system is designed as a complete and deployable application. It integrates multiple components, including a Flutter-based user interface, an AI inference layer using TensorFlow Lite, and modular service architecture for speech and interaction features. This system-level integration ensures that the application is not limited to theoretical evaluation but is capable of real-world usage.

While the achieved accuracy of approximately 75% is moderate compared to high-end CNN-based systems, it reflects realistic performance given the constraints of dataset size, static gesture recognition, and mobile deployment requirements. The system is therefore positioned as a practical accessibility solution rather than a computationally intensive experimental model. Additional features such as sign-to-text, speech-to-text, text-to-speech, and learning modules further enhance its usability and real-world relevance.

1.6 Methodology and Software Lifecycle for This Project

The development of the Sign Bridge application follows the Agile Software Development Methodology, combined with an iterative process model. This approach was selected due to

the experimental and evolving nature of machine learning-based systems, where continuous refinement, testing, and improvement are essential. Agile methodology supports incremental development, allowing features to be implemented, tested, and improved in cycles rather than being developed in a single rigid phase.

The system consists of multiple interconnected components, including the mobile application interface, gesture recognition pipeline, on-device machine learning inference layer, speech processing services, and data handling mechanisms. Each of these components was developed incrementally across multiple iterations, ensuring that individual modules were functional and stable before being integrated into the complete system.

The overall development process followed a structured sequence of activities, while still maintaining flexibility for iteration and improvement:

- Requirement identification and analysis based on accessibility and communication needs
- Dataset collection and preparation for gesture classification
- Feature extraction using Media Pipe to obtain hand landmarks
- Development of gesture classification models and optimization for mobile deployment (converted into TensorFlow Lite format)
- Implementation of real-time inference pipeline using on-device model (**sign_model.tflite**)
- Integration of core modules into the Flutter-based mobile application
- Development of additional features such as speech-to-text, text-to-speech, dictionary, and daily challenge modules
- Testing, validation, and performance optimization across different devices and conditions

Unlike traditional approaches that rely on heavy machine learning models such as Random Forest or MLP for deployment, the Sign Bridge system focuses on lightweight on-device inference using TensorFlow Lite. The gesture recognition pipeline includes landmark normalization [2](conversion of 63 features into a structured format), followed by efficient real-time classification. This ensures that the system remains responsive and suitable for mobile environments.

The iterative nature of Agile development allowed continuous improvements throughout the project lifecycle. Feedback from testing phases was used to refine model performance,

optimize landmark preprocessing techniques, improve inference stability, and enhance the user interface design. Additionally, real-time constraints such as inference throttling (~300ms) and UI responsiveness were fine-tuned during iterations to achieve a balance between performance and usability.

Overall, the adoption of Agile methodology and an iterative development model enabled the Sign Bridge system to evolve into a stable, efficient, and user-friendly application. This approach ensured that both technical performance and user experience were continuously improved, resulting in a practical and deployable assistive communication solution.

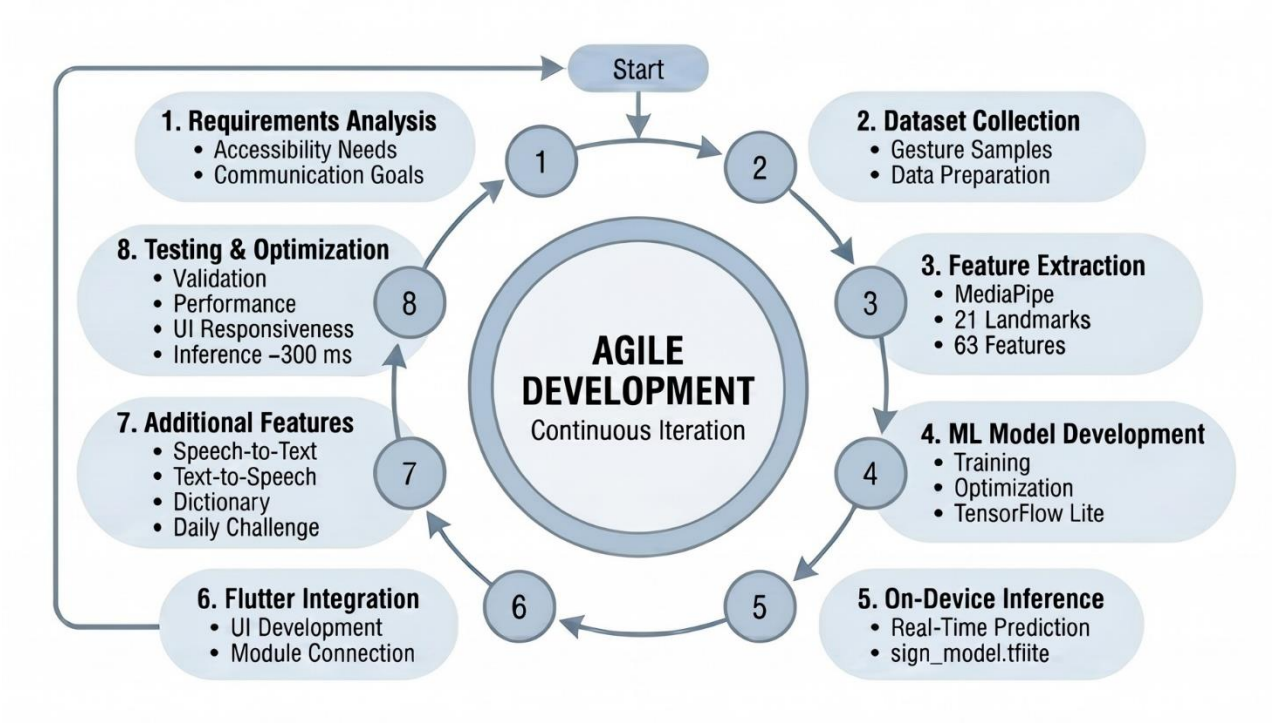

Figure 1-1 SDLC

1.7 Rationale behind Selected Methodology

The selection of the Agile Software Development Methodology is strongly justified by the nature of machine learning-based system development. Unlike traditional software systems, where outputs are deterministic and follow predefined logic, machine learning systems depend heavily on factors such as data quality, feature representation, model architecture, and parameter tuning. The performance of such systems cannot be finalized in a single development phase, as it requires continuous experimentation, evaluation, and refinement. Therefore, a rigid and sequential development model such as the Waterfall approach is not well-suited for this type of project.

Agile methodology, on the other hand, supports incremental development and continuous improvement, making it highly suitable for projects like Sign Bridge. Each development iteration provides an opportunity to evaluate system performance, particularly in critical components such as gesture recognition accuracy, real-time inference stability, and speech processing reliability. Based on the results of each iteration, adjustments can be made to improve model behavior, including fine-tuning preprocessing techniques, optimizing landmark normalization, and enhancing prediction consistency.

In addition to machine learning refinement, Agile also facilitates improvements in the overall system design, particularly the user interface and user experience. Since Sign Bridge is designed as an accessibility-focused mobile application, usability plays a crucial role in its effectiveness. Iterative feedback allows continuous enhancement of UI components, navigation flow, and real-time interaction elements, ensuring that the system remains intuitive and user-friendly.

Furthermore, integrating machine learning models into a mobile environment introduces additional challenges, such as performance optimization, resource management, and real-time responsiveness. The use of on-device inference (via TensorFlow Lite) requires careful tuning to ensure that the application runs efficiently without causing delays or excessive resource consumption. Agile methodology allows these optimizations to be carried out progressively, with testing and adjustments performed at each stage of development.

Another important advantage of Agile is its ability to maintain flexibility while still supporting structured planning and documentation. Even though development occurs in iterations, each phase is guided by clearly defined goals, ensuring that the system evolves in a controlled and organized manner. This balance between flexibility and structure is particularly beneficial in projects that combine multiple domains, such as mobile development, computer vision, and machine learning.

Chapter 2
Problem Definition

2 Problem Definition

This chapter explains the precise problem addressed by the project "**Sign Bridge: AI-Based Sign Language Translation and Communication System**." It highlights the communication challenges faced by deaf and mute individuals, particularly in situations where the surrounding people are not familiar with sign language. These challenges are especially significant in real-world environments such as educational institutions, healthcare facilities, workplaces, and public interaction spaces, where effective communication is essential.

The chapter further defines the expected outcomes of the project, including real-time sign-to-text translation, speech-based interaction, and user-friendly accessibility features. It also outlines the major deliverables, such as the mobile application, on-device machine learning model, and integrated communication modules. In addition, the development requirements both technical and functional are discussed to provide a clear understanding of the system's scope.

2.1 Problem Statement

Deaf and mute individuals face significant communication barriers in their daily lives, particularly in environments where sign language is not widely understood. In Pakistan, the primary communication medium used by the deaf community is Pakistan Sign Language (PSL). However, the general population has limited familiarity with PSL, resulting in a persistent communication gap during everyday interactions. This gap affects multiple aspects of life, including education, healthcare, employment, social participation, and access to essential public services.

In educational environments, deaf students often encounter difficulties in communicating with teachers and classmates who do not understand sign language. This limits their participation in classroom discussions and affects their learning outcomes. Similarly, in healthcare settings, ineffective communication between patients and medical professionals can lead to misunderstandings, incorrect diagnoses, or delays in treatment. In professional environments, deaf individuals may face exclusion from meetings, teamwork activities, and workplace communication, reducing their opportunities for growth and inclusion. These challenges extend beyond inconvenience and directly impact independence, confidence, and overall quality of life.

The most common approach to addressing this communication gap is the use of human interpreters. While interpreters play a crucial role in facilitating communication, their availability is limited, and their services can be costly. Moreover, interpreters are not accessible in many routine situations, such as shopping, traveling, or interacting in public spaces. As a result, deaf and mute individuals are often unable to communicate effectively in real-time scenarios without external support. This widely used alternative is written communication, such as exchanging notes or typing messages on mobile devices. Although this method can be useful in certain contexts, it is inherently slow and disrupts the natural flow of conversation. It is also less effective in situations requiring immediate interaction. Additionally, not all individuals are equally comfortable with written language, particularly those with limited access to formal education, making this approach less inclusive.

From a technological perspective, several sign language recognition systems have been developed; however, many of them are primarily designed for [3] American Sign Language (ASL) and do not account for regional variations such as PSL. Furthermore, a large number of these systems rely on computationally intensive deep learning techniques, which require high-end hardware and GPU support. This limits their applicability in real-world settings, particularly in developing regions where users typically rely on mid-range mobile devices with limited processing capabilities. Another limitation of existing systems is that they often support only one-directional communication. Most applications focus on converting sign gestures into text, but they do not provide reverse functionality, such as converting text or speech into sign representations. Since communication is inherently bidirectional, the absence of this capability reduces the practical usefulness of such systems. Users require a solution that allows them not only to express themselves but also to understand responses from others.

The proposed Sign Bridge system aims to address these challenges by providing a mobile-based sign language translation solution that supports real-time interaction. The system recognizes predefined hand gestures using landmark-based machine learning techniques and converts them into text output. It also includes additional features such as speech-to-text, text-to-speech, and educational modules, enabling a more comprehensive communication experience. Although the system does not fully solve the complex problem of complete sign language translation, it provides a practical and scalable foundation for real-time gesture-based communication. By focusing on accessibility, efficiency, and mobile deployment, the proposed solution contributes toward reducing the communication gap and improving interaction between deaf individuals and the general population.

2.2 Deliverables and Development Requirements

The final deliverable of this project is a working mobile-based sign language translation system that recognizes selected hand gestures and converts them into text and optional speech output. The system is designed as a practical software solution rather than only a machine learning experiment. Therefore, the deliverables include both the trained machine learning components and the user-facing mobile application.

The primary deliverable is the mobile application interface developed using Flutter. This interface allows users to access the sign recognition feature, view translated text, use optional audio output, access the sign dictionary, and interact with learning and practice-related features. Since the target users include individuals with hearing or speech impairments, the user interface is designed to remain simple, clear, and easy to navigate. The application presents output in a readable format and avoids unnecessary complexity.

The second important deliverable is the gesture recognition module. This module uses the device camera to capture the user's hand gesture and applies Media Pipe to detect 21 hand landmarks. Each landmark contains x, y, and z coordinate values, resulting in 63 numerical features representing a single hand gesture. These features are preprocessed and normalized before being passed to the on-device machine learning model for classification. The recognized sign label is then displayed on the screen in real time.

The third deliverable is the trained machine learning model deployed in a mobile-compatible format. Instead of using .pkl or .pth files in the final application, the trained model is converted into a TensorFlow Lite format (sign_model.tflite) for efficient on-device inference. This model is responsible for predicting the correct sign label based on the processed landmark feature vector and is optimized for real-time performance on mobile devices.

The system also includes a sign dictionary and learning support module. This module provides users with a list of supported signs and their meanings. It helps users understand which gestures are recognized by the system and supports basic learning. A practice or challenge mode allows users to perform signs and observe the system’s prediction, reinforcing learning through interaction. The development requirements of the project include both hardware and software components. From the hardware perspective, a development laptop or desktop is required for dataset preparation, model training, and application development. A smartphone with a functional camera is required for real-time testing and deployment. Since the system relies on

hand landmark detection, camera quality and lighting conditions can influence recognition performance.

From the software perspective, the project requires Python for initial model development and data preprocessing, Media Pipe for hand landmark extraction, and TensorFlow Lite for deploying the trained model on mobile devices. The mobile application is developed using Flutter, with additional libraries such as tflite_flutter, speech_to_text, and [4]flutter_tts for AI inference and speech functionalities. Development tools such as Visual Studio Code, Android Studio, and Git are used for coding, testing, and version control throughout the project lifecycle.

The main project deliverables can be summarized as follows:

Table 2.1 Deliverables of the Sign Language

Deliverable	**Description**
Mobile Application	Flutter-based mobile interface for real-time sign recognition, text output display, speech features, and learning modules
Gesture Recognition Module	Camera-based hand landmark extraction using MediaPipe with 21-point hand detection
Machine Learning Models	TensorFlow Lite (.tflite) model (sign_model.tflite) used for on-device gesture classification
Dataset	Labeled landmark-based dataset containing 63 input features (21 landmarks × x, y, z) per record
Sign Dictionary	Module containing supported sign labels, descriptions, and visual references for learning
Practice Module	Interactive feature including practice mode and daily challenges for improving gesture recognition accuracy
Speech Modules	Speech-to-text and text-to-speech integration for bidirectional communication support
Documentation	Final report, SRS-based requirements, system design, implementation details, and testing results

Table 2.1 provides a comprehensive overview of the core software modules, data assets, and academic documentation that comprise the complete system. It clearly maps each major project milestone, from the front-end user interface to the back-end machine learning components, ensuring a structured trace of the final year project's execution.

The major development requirements can be summarized as follows:

Table 2.2 Development Requirements

Requirement Type	Description
Operating System	Windows/Linux for development; Android device for testing
Programming Languages	Python, Dart
ML Libraries	TensorFlow Lite (for on-device inference), TensorFlow (for model conversion and optimization)
Computer Vision Tool	MediaPipe
Mobile Framework	Flutter
Development Tools	VS Code, Android Studio, Git
Hardware	Laptop/PC, smartphone camera
Model Files	TensorFlow Lite model (.tflite) for mobile inference and labels.json for class mapping
Dataset Type	Supervised labeled dataset containing hand landmark feature vectors (63 features per sample)
Problem Type	Multiclass classification

Table 2.2 shows the technical requirements table outlines the complete hardware, software, and algorithmic ecosystem required to develop and run the system. It establishes a clear baseline for the development environment, development tools, and data structures necessary to support the cross-platform machine learning application.

2.3 Current System

At present, communication between deaf and mute individuals and hearing individuals is mostly handled through traditional methods. The most common method is the use of human interpreters who understand sign language and can translate between signers and non-signers. Although interpreters are effective, they are not always available. In many situations, deaf and mute individuals cannot depend on interpreters because of cost, location, time, or privacy

concerns. Additionally, interpreter-based communication is not scalable for everyday interactions, especially in informal or spontaneous environments.

Another common method is written communication. A deaf or mute person may write a message on paper or type it on a mobile phone to communicate with a hearing person. While this method can work for simple conversations, it is slow and does not support natural real-time interaction. It may also be difficult in urgent situations where quick communication is required. Written communication also assumes that both individuals are comfortable with the same written language, which may not always be true, particularly in regions with varying literacy levels.

Some people use mobile messaging applications or note-taking applications as an alternative. However, these tools do not understand sign language and only provide a medium for text-based communication. As a result, they do not address the core problem of translating hand gestures into a form that can be understood by non-signers. They also do not assist hearing individuals in interpreting or learning sign gestures, limiting their usefulness in accessibility-focused scenarios.

Existing sign language translation applications and research systems also have notable limitations. Many of these systems are developed for American Sign Language (ASL) or other internationally recognized sign languages. Since sign languages vary significantly across regions, systems designed for ASL may not be directly applicable to Pakistan Sign Language (PSL). Differences in gesture patterns, grammar, and semantics create a gap that reduces the effectiveness of such solutions for local users.

Hardware-based systems have also been explored in research, including glove-based recognition systems that use sensors to track finger movement, hand orientation, and motion. While these systems can provide accurate results, they are not practical for everyday use because they require users to wear specialized hardware. This increases cost, reduces convenience, and limits widespread adoption, especially in low-resource environments.

Some advanced systems rely on deep learning techniques, such as Convolutional Neural Networks (CNNs) or recurrent models, to process raw images or video frames. Although these approaches can achieve high accuracy in controlled conditions, they require large datasets, high computational power, and often GPU support. This makes them less suitable for deployment on standard mobile devices, particularly when real-time processing is required.

The current communication environment therefore lacks a simple, mobile-based, low-cost, and locally adaptable sign recognition tool that can function effectively in real-world conditions. The proposed Sign Bridge system improves upon the current situation by utilizing a standard smartphone camera combined with Media Pipe-based hand landmark detection. Instead of processing raw images, the system extracts 21 hand landmarks and converts them into 63 numerical features, significantly reducing computational complexity.

These features are then processed using an optimized on-device inference model implemented through TensorFlow Lite, enabling real-time gesture recognition directly on the mobile device. In addition to sign-to-text translation, the system also integrates speech-to-text, text-to-speech, and learning modules, making it a more comprehensive and practical communication solution. This approach enhances efficiency, reduces dependency on external resources, and ensures better suitability for mobile deployment and real-time usage.

Table 2.3 Comparison of Current Systems

Existing Approach	Limitation	Proposed System Improvement
Human Interpreter	Not always available, may be costly	Mobile-based translator available anytime
Written Communication	Slow and unnatural	Real-time gesture-to-text conversion
ASL-based Applications	Not suitable for local sign variations	Can be trained on selected local sign labels
Sensor Gloves	Expensive and hardware-dependent	Uses only smartphone camera
Heavy Deep Learning Models	Require high computation	Uses lightweight landmark-based ML models
General Note Apps	Do not recognize signs	Recognizes gestures and outputs labels

Table 2.3 shows comparative analysis table establishes the project's core research justification by contrasting existing communication methods against the engineered solution. It clearly pairs the operational limitations of traditional tools with the specific technological enhancements provided by the proposed system.

Overall, the current system of communication for deaf and mute individuals is fragmented and limited. The proposed project provides a more practical alternative by combining computer vision, supervised machine learning, and mobile application development into a single

accessible system. While the system currently supports a limited set of signs and achieves moderate accuracy, it establishes a strong foundation for future expansion into a larger and more accurate sign language translation platform.

Chapter 3
Requirement Analysis

3 Requirement Analysis

This chapter describes the functional and non-functional requirements of the Sign Bridge: AI-Based Sign Language Translation and Communication System. Requirement analysis is a critical stage in software development because it defines what the system is expected to do and how users will interact with it. It establishes a clear understanding of system behavior, user expectations, and performance criteria, ensuring that all components of the application are aligned with the intended objectives.

The chapter includes use case diagrams, detailed use cases, functional requirements, and non-functional requirements that collectively describe the complete behavior of the system. These requirements cover all major modules of the Sign Bridge application, including real-time sign-to-text translation, text-to-sign conversion, speech-to-text input, text-to-speech output, and learning features such as the dictionary and daily challenge module. Each requirement is defined in a structured manner to ensure clarity and traceability throughout the development process.

3.1 Use Case Diagram

A use case diagram provides a high-level view of system interactions by identifying the main actors and the services they access. In the proposed system, the primary users include:

- Mute User
- Deaf User
- Normal (Hearing) User

Although all three actors interact with similar modules, their purposes differ slightly based on communication needs.

The **Mute User** primarily uses the system to:

- Perform sign gestures
- Convert signs into text
- Generate optional audio output
- Practice sign accuracy

The **Deaf User** primarily uses the system to:

- Convert text into sign

- View dictionary entries
- Learn and practice signs

The **Normal User** primarily uses the system to:

- Convert speech or text into sign
- Learn sign language using the dictionary
- Communicate with deaf or mute individuals

The use case diagram illustrates the interaction between these actors and system modules.

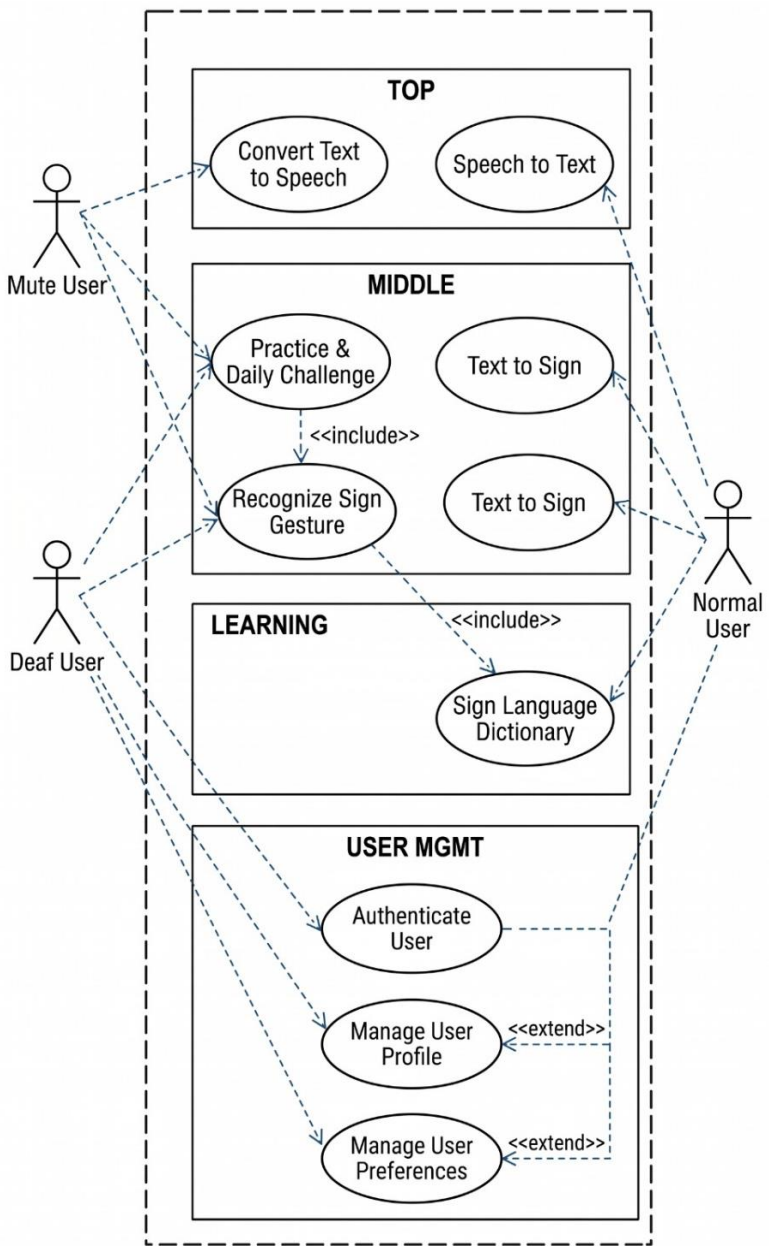

Figure 3-1 Use Case Diagram

The diagram shows that gesture recognition is the core functionality of the system, while dictionary and practice modules support learning and improvement. Authentication is optional but required for storing user preferences and progress.

3.2 Detailed Use Cases

Detailed use cases explain the step-by-step interaction between actors and the system. They describe triggers, preconditions, normal flow, alternative flow, and postconditions.

3.2.1 Authenticate User (UC-1)

This use case allows users to launch and access the application's features. Authentication is optional and not required for core functionality. Upon startup, the system initializes services such as model loading, asset mapping, and theme configuration.

Table 3.1 Use Case: Authenticate User

Element	Description
Use Case ID	UC-1
Use Case Name	Authenticate User
Primary Actor	Mute / Deaf / Normal User
Description	Allows user to log in, register, or continue as guest
Trigger	User opens application
Preconditions	Application installed and running
Postconditions	User session created (if logged in) or guest mode activated
Normal Flow	Open app → Select login/register → Enter credentials → Validate → Access dashboard
Alternative Flow	Invalid credentials → Display error

Table 3.1 shows the use case specification table maps the structural parameters and operational logic governing the initial user authentication module. It establishes the clear boundaries, behavioral paths, and system conditions required to securely onboard users into the application environment.

3.2.2 Recognize Sign Gesture (Sign → Text)

This is the core AI functionality of Sign Bridge. The system captures live camera input, extracts 21 hand landmarks (x, y, z coordinates), normalizes 63 feature values, and performs

TensorFlow Lite inference to classify the gesture. The predicted label is displayed as text in real time. Inference is throttled to ensure stable performance.

Table 3.2 Use Case: Recognize Sign Gesture

Element	Description
Use Case ID	UC-2
Use Case Name	Recognize Sign Gesture
Primary Actor	Mute User, Deaf User
Description	Captures hand landmarks and predicts sign label
Trigger	User activates recognition mode
Preconditions	Camera functional and ML model loaded
Postconditions	Recognized sign displayed as text
Normal Flow	Open camera → Detect landmarks → Extract 63 features → Predict label → Display text
Alternative Flow	Hand not detected → Show warning
Business Rules	Prediction shown only if confidence threshold met

Table 3.2 shows the use case specification table explicitly details the structural logic, hardware dependencies, and algorithmic flows governing the sign gesture recognition module.

3.2.3 Translate Text to Speech

This use case enables the system to convert textual content into spoken audio using the device's Text-to-Speech (TTS) engine. The text may originate from recognized sign gestures or manual user input. The user must explicitly trigger audio playback. This feature assists mute users in verbal communication with others.

Table 3.3 Use Case: Translate Text to Audio

Element	Description
Use Case ID	UC-3
Use Case Name	Convert Text to Speech
Primary Actor	Mute User
Description	Converts text into spoken audio using platform TTS engine.

Trigger	User taps “Speak” or “Play Audio” button.
Preconditions	Text must be available.
Postconditions	Audio output generated through device speaker.
Normal Flow	1. Text available (recognized or typed). 2. User taps Speak button. 3. System sends text to flutter_tts engine. 4. Audio playback begins.
Business Rules	Audio playback must always be user-triggered.

Table 3.3 shows the use case specification table maps out the functional requirements, architectural logic, and execution steps for the text-to-speech synthesis engine.

3.2.4 Sign Language Dictionary

This use case allows users to browse and search supported sign gestures. Each entry includes the visual sign asset and its corresponding meaning. The dictionary supports self-learning and reference, enhancing accessibility and educational value.

Table 3.4 Use Case: Sign Dictionary

Element	Description
Use Case ID	UC-4
Use Case Name	Sign Language Dictionary
Primary Actor	All Users
Description	Displays list of supported signs and meanings
Trigger	User opens dictionary
Preconditions	Dictionary data available
Postconditions	Sign meaning displayed

Table 3.4 show the use case specification table maps out the structural attributes and behavioral requirements of the built-in Sign Language Dictionary module. It provides a formal breakdown of the triggers, preconditions, and postconditions that govern how users browse and access the application's vocabulary data.

3.2.5 Practice Sign & Accuracy Testing

This use case provides an interactive learning module where users practice sign gestures. The system generates a daily set of sign prompts using a deterministic date-based shuffle. The user performs the gesture, and the AI model evaluates it by predicting the sign label. Feedback is provided to reinforce learning and improve accuracy.

Table 3.5 Use Case: Practice Sign & Accuracy

Element	Description
Use Case ID	UC-5
Use Case Name	Practice Sign
Primary Actor	Mute User, Learning User
Description	Compares user gesture with trained model
Trigger	User selects practice mode
Preconditions	Camera and model active
Postconditions	Accuracy score displayed
Normal Flow	Select sign → Perform gesture → Extract features → Predict → Show accuracy

Table 3.5 shows the use case specification table maps out the operational parameters, hardware requirements, and procedural steps for the interactive sign practice module.

3.2.6 Manage User Preferences

This use case enables users to configure application-level settings such as theme mode (light/dark) and interface preferences. Preferences are stored locally on the device and applied immediately using Provider-based state management.

Table 3.6 Use Case: Manage User Preferences

Element	Description
Use Case ID	UC-6
Use Case Name	Manage User Preferences
Primary Actor	Mute User, Deaf User, Normal User
Description	Allows authenticated users to view and modify application settings and preferences.

Trigger	User selects “Settings” or “Preferences” from the dashboard.
Preconditions	User must be authenticated (logged in).
Postconditions	Updated preferences are saved and applied to the system.
Normal Flow	1. User logs into the system. 2. User opens the Preferences section. 3. System displays current settings. 4. User modifies one or more settings. 5. User saves changes. 6. System validates and stores updated preferences.
Alternative Flow	A1: Invalid input → System displays error message. A2: Save failure → System prompts user to retry.
Business Rules	Preferences are user-specific and must be stored securely.
Priority	Medium
Frequency of Use	Occasional

Table 3.6 shows the use case specification table maps out the administrative parameters, security conditions, and error-handling flows for the user preferences management module. It provides developers with a structured blueprint for implementing secure, user-specific application settings and data-saving rules.

3.2.7 Manage User Profile

The Manage User Profile use case supports personalization and account maintenance within the system. Since the application allows optional authentication, profile management is restricted to registered users only. This module ensures that user-specific information such as name, email, and preferences can be updated securely. Proper validation mechanisms are implemented to prevent incorrect or unauthorized modifications. The feature enhances usability and supports long-term interaction with the system.

Table 3.7 Use Case: Manage User Profile

Element	**Description**
Use Case ID	UC-7

Use Case Name	Manage User Profile
Primary Actor	Mute User, Deaf User, Normal User
Description	Allows authenticated users to view and update their profile information.
Trigger	User selects “Profile” from the application dashboard.
Preconditions	User must be logged into the system.
Postconditions	Updated profile information is stored successfully in the system.
Normal Flow	1. User logs into the system. 2. User navigates to the Profile section. 3. System displays existing profile information. 4. User edits selected fields (e.g., name, email). 5. User submits updated information. 6. System validates input data. 7. System saves changes and displays confirmation message.
Alternative Flow	A1: Invalid input data → System displays validation error. A2: Database error → System displays save failure message.
Business Rules	Profile changes must apply only to the authenticated user. Sensitive data must be protected.
Priority	Medium
Frequency of Use	Occasional

Table 3.7 shows the use case specification table maps out the functional requirements, security boundaries, and error-handling paths for the user profile management module. It acts as a technical blueprint for developers to implement secure, isolated, and validated account editing features within the application.

3.2.8 Daily Challenge

This use case provides an interactive learning module where users practice sign gestures. The system generates a daily set of sign prompts using a deterministic date-based shuffle. The user

performs the gesture, and the AI model evaluates it by predicting the sign label. Feedback is provided to reinforce learning and improve accuracy.

Table 3.8 Use Case: Daily Challenge

Element	**Description**
Use Case ID	UC-8
Use Case Name	Daily Challenge
Primary Actor	Learning User
Description	Presents a daily set of sign challenges based on deterministic date-seeded shuffle.
Trigger	User opens Daily Challenge module.
Preconditions	Model and sign assets loaded.
Postconditions	User receives feedback on performance.
Normal Flow	1. System generates date-seeded letter set. 2. Sign image displayed as prompt. 3. User performs gesture. 4. Model predicts gesture. 5. System compares expected vs predicted label. 6. Feedback displayed.
Alternative Flow	A1: Incorrect gesture → System prompts retry.
Business Rules	Daily set must remain consistent for the same calendar date.

Table 3.8 shows the use case specification table maps out the algorithmic parameters, operational paths, and processing logic for the Daily Challenge module.

3.3 Functional Requirements

Functional requirements describe the specific services and behaviors that the Sign Bridge system must provide. These requirements define what the system is expected to do from the user's perspective. Sign Bridge is a Flutter-based assistive communication system that integrates real-time hand landmark detection, on-device TensorFlow Lite inference, speech recognition, speech synthesis, and learning support modules. The functional requirements

focus on real-time AI inference, multimodal translation (sign, speech, text), and interactive user experience.

The system must capture live video input using the mobile device camera. It shall detect 21 hand landmarks per frame using a hand landmark detection plugin. Each landmark contains three spatial coordinates (x, y, z), forming a 63-dimensional feature vector. The system must normalize the landmarks by subtracting the wrist reference point and scaling values before passing them to the TensorFlow Lite classification model (sign model. tflite).

The trained TF Lite model shall perform multiclass classification and output the predicted sign label. The system must display the recognized label on the user interface in near real time. Inference shall be throttled (approximately 300ms intervals) to ensure stable and efficient performance. In addition to gesture recognition, the system shall support text-to-speech functionality. After a sign is recognized or text is manually entered, the user may press a “Speak” or “Play Audio” button to trigger speech output. Audio playback must always be user initiated.

The system shall also provide a speech-to-text module. When the user activates the microphone, spoken audio must be transcribed into text in real time using the device’s speech recognition service. The system must provide a Text-to-Sign module. When text is entered, each character shall be mapped to corresponding sign image assets stored locally. The mapped sequence must be displayed visually to support communication with deaf users. The system shall include a Sign Language Dictionary and Learning Hub. This module must display supported signs and their meanings using locally stored assets. Users shall be able to browse and search available signs.

The system shall provide a Daily Challenge / Practice module. The application must generate a deterministic daily set of signs based on the current date. Users shall perform gestures that are evaluated using the same TF Lite inference pipeline. Feedback shall be displayed to support learning. User authentication shall remain optional. Core features (sign recognition, speech-to-text, text-to-sign, dictionary, and practice mode) must function without requiring login. If authentication is enabled, the system shall allow profile and preference management. User preferences such as theme selection (light/dark mode) and interface settings must be stored locally and applied immediately using Provider-based state management.

The key functional requirements are summarized below.

Table 3.9 Functional Requirements

Req ID	Functional Requirement	Priority
FR-1	The system shall capture real-time hand gestures using the device camera	High
FR-2	The system shall detect 21 hand landmarks per frame	High
FR-3	The system shall convert landmarks into a normalized 63-dimensional feature vector	High
FR-4	The system shall classify gestures using a TensorFlow Lite model	High
FR-5	The system shall display the predicted sign label in real time	High
FR-6	The system shall throttle inference to ensure stable performance	High
FR-7	The system shall provide optional text-to-speech output	Medium
FR-8	The system shall provide speech-to-text transcription	High
FR-9	The system shall convert text into corresponding sign image assets	High
FR-10	The system shall provide a sign language dictionary module	High
FR-11	The system shall provide a daily challenge/practice mode with feedback	Medium
FR-12	The system shall allow optional user login	Low
FR-13	The system shall allow management of user preferences (e.g., theme mode)	Medium
FR-14	The system shall store preferences locally on the device	Medium

Table 3.9 shows the functional requirements table provides an exhaustive index of the core operational capabilities and technical mandates that define the system's software architecture.

It systematically categorizes every required behavior, from real-time computer vision tracking to user data management, prioritizing them by implementation urgency.

3.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes and performance characteristics of the Sign Bridge system. Unlike functional requirements, which define what the system does, non-functional requirements specify how effectively and efficiently the system performs its operations. Since Sign Bridge operates in a real-time mobile environment and integrates on-device AI inference, performance, usability, and reliability are critical to ensuring smooth interaction for users, particularly individuals with hearing or speech impairments.

3.4.1 Performance

The system must perform real-time gesture recognition with minimal latency. After detecting hand landmarks and normalizing the 63-dimensional feature vector, the TensorFlow Lite model must generate a prediction within an acceptable response time. Under normal operating conditions, prediction results should appear within 300–500 milliseconds. Inference is intentionally throttled (~300ms intervals) to maintain stability and prevent excessive resource usage. Speech-to-text transcription should also display results with minimal delay to support natural communication flow.

3.4.2 Usability

Usability is a core requirement because the system targets mute, deaf, and learning users.

The user interface must:

- Be simple and intuitive
- Use clearly labeled buttons and readable fonts
- Maintain visual consistency through centralized theming (App Theme, App Colors, App Text Styles)
- Support light and dark modes
- Avoid complex navigation steps

Critical functions such as microphone activation and audio playback must be clearly user-triggered to avoid confusion.

3.4.3 Reliability

The system must operate reliably during camera streaming, landmark detection, and TF Lite inference. It must not crash during:

- Continuous camera preview
- Real-time landmark extraction
- Model loading or prediction
- Speech recognition sessions

The prediction pipeline must produce consistent outputs for similar input gestures under stable lighting conditions.

Graceful error handling must be implemented for:

- No hand detected
- Model loading failure
- Permission denial (camera/microphone)

3.4.4 Accuracy

Although perfect recognition cannot be guaranteed due to lighting conditions, occlusion, and hand positioning variations, the classification model must achieve acceptable performance. The deployed TensorFlow Lite model achieves approximately 75% accuracy on the test dataset, exceeding the minimum baseline requirement of 70% accuracy. Accuracy may vary in real-world conditions; therefore, confidence thresholds are used to reduce unstable predictions.

3.4.5 Security and Privacy

Sign Bridge follows a local-first architecture. Core functionality does not require external servers or cloud storage.

If authentication or profile storage is enabled:

- User information must be stored securely.
- Sensitive data must not be exposed.
- Access must be restricted to the authenticated user only.

Camera and microphone access must require explicit user permission in accordance with Android platform security policies.

3.4.6 Maintainability

The system architecture is designed using a feature-based modular structure with clean separation of responsibilities:

- features/... → UI and feature logic
- core/services/... → reusable services (AI, speech, asset mapping)
- core/providers/... → state management
- core/theme/... → centralized theming

This modular structure ensures that:

New sign classes can be added by updating the model and label file.

The TF Lite model can be replaced without major UI changes.

Additional features can be integrated with minimal architectural modification.

3.4.7 Portability

The application must run on standard Android devices without requiring specialized hardware. Since the system uses on-device TensorFlow Lite inference and platform speech services, it relies only on:

- Device camera
- Microphone
- Speaker
- Standard Android runtime environment
- No external AI hardware or dedicated servers are required.

3.4.8 Scalability

The system should support:

- Adding new sign classes by retraining and replacing the TF Lite model.
- Expanding the dictionary module with additional sign assets.
- Enhancing AI performance without restructuring the entire application.

The modular AI inference service allows future integration of improved models or optimization techniques.

The main non-functional requirements are summarized below.

Table 3.10 Non-Functional Requirements

Req ID	Non-Functional Requirement	Description
NFR-1	Usability	Interface must be simple, accessible, and intuitive
NFR-2	Performance	Gesture prediction must occur within 300–500ms
NFR-3	Accuracy	Model must achieve at least 70% test accuracy (current ~75%)

NFR-4	Reliability	System must operate without crashes during camera and inference usage
NFR-5	Security	User data and permissions must be handled securely
NFR-6	Portability	Application must run on standard Android devices
NFR-7	Maintainability	Modular architecture must support future updates
NFR-8	Scalability	System must support addition of new sign classes and features

Table 3.10 shows the non-functional requirements table outlines the critical quality metrics, performance baselines, and architectural constraints that govern the system beyond its basic functionality. It sets measurable standards for usability, processing speed, classification accuracy, and security to ensure the application is viable, secure, and production-ready.

Chapter 4
Design and Architecture

4 Design and Architecture

This chapter presents the architectural design of the Sign Language Translator for Deaf and Mute People Using Machine Learning. It describes the overall system structure, data representation, process flow, and formal design models. The purpose of this chapter is to explain how the various components of the system are organized and how they interact with each other to deliver gesture recognition and translation functionality.

The design decisions made in this chapter are guided by the functional and non-functional requirements identified in Chapter 3. The system follows a modular layered architecture that separates the user interface, application logic, machine learning inference, and data storage into distinct layers. This separation ensures maintainability, scalability, and ease of future extension.

4.1 System Architecture

The Sign Language Translator system is designed using a four-layered modular architecture. This architectural style ensures that each component of the system has a well-defined responsibility and interacts with other components through clear interfaces. Since the system combines mobile application development, computer vision, and machine learning, a layered approach is the most appropriate structure for organizing these responsibilities.

The four layers of the system are:

1. Presentation Layer
2. Application Layer
3. Machine Learning Processing Layer
4. Data Layer

Each layer is described in detail below.

4.1.1 Presentation Layer

The presentation layer is the topmost layer of the system and represents the user-facing interface. It is developed using Flutter, which enables cross-platform compatibility on Android and iOS devices. The presentation layer is responsible for capturing user input, displaying recognition results, and providing navigation between application modules.

The main screens included in this layer are:

- **Sign Recognition Screen**: Displays live camera feed and predicted sign label

- **Dictionary Screen**: Lists supported signs and their meanings
- **Practice Screen**: Allows users to practice gestures and view accuracy feedback
- **Profile Screen**: Allows authenticated users to view and update profile information
- **Preferences Screen**: Allows users to configure application settings
- **Authentication Screen**: Supports optional user login and registration

The presentation layer does not perform any machine learning computation. It only collects input from the user and displays output received from the lower layers. This separation ensures that the user interface remains responsive and independent of processing complexity.

4.1.2 Application Layer

The application layer sits below the presentation layer and manages the business logic of the system. It acts as the coordinator between user interaction, machine learning inference, and data management. When the user initiates a gesture recognition session, the application layer receives the camera feed, passes it to the machine learning processing layer, and returns the prediction result to the presentation layer.

The main responsibilities of this layer include:

- Managing camera stream initialization
- Coordinating MediaPipe landmark extraction
- Preparing feature vectors for model input
- Handling authentication and session management
- Managing navigation between modules
- Handling error conditions and user notifications

The application layer ensures that all system modules communicate in a structured and controlled manner.

4.1.3 Machine Learning Processing Layer

The machine learning processing layer is the core intelligence of the system. It is responsible for extracting hand landmarks, processing features, and classifying gestures into sign labels. This layer consists of two main components:

Media Pipe Landmark Detector:

Media Pipe is used to detect 21 hand landmarks from each camera frame. Each landmark contains x, y, and z coordinate values. With 21 landmarks and 3 values per landmark, each frame produces a 63-dimensional feature vector. This vector serves as the input to the classification models.

Machine Learning Classifiers:

Two models are implemented and compared:

- **Random Forest Classifier**: A classical ensemble learning model that uses multiple decision trees for classification. It is saved in .pkl format using [5]Scikit-learn.
- **Multi-Layer Perceptron (MLP)**: A feedforward neural network implemented using PyTorch. It is saved in .pth format.

Both models are trained offline on a labeled dataset and loaded into the application during runtime for inference. The predicted sign label and confidence score are returned to the application layer for display.

4.1.4 Data Layer

The data layer manages all persistent information within the system. This includes user profile data, preferences, and recognition history. Since the system emphasizes privacy, all gesture processing is performed locally on the device. The data layer stores user-related information using structured local storage mechanisms.

The main data entities handled by this layer are:

- **User:** Stores name, email, and account information
- **Preferences**: Stores user-specific settings
- **Recognition History**: Stores previously recognized signs (optional)
- **Sign Dictionary**: Stores predefined sign labels and meanings

The data layer ensures that user-specific information is retrieved and stored securely.

4.1.5 System Architecture Summary

The overall architecture of the Sign Language Translator system can be summarized as follows:

Table 4.1 System Architecture Layers

Layer	Technology	Responsibility
Presentation Layer	Flutter	User interface, input capture, output display
Application Layer	Dart / Flutter Logic	Business logic, session management, coordination
ML Processing Layer	Hand Landmark Plugin, TensorFlow Lite, Explicit normalization pipeline	Landmark detection, feature extraction, classification
Data Layer	Local Storage	User data, preferences, history

Table 4.1 shows the architectural design table provides a structured, multi-tier overview of the system's software ecosystem, broken down by functional responsibilities. It maps out how different layers of technology ranging from the Flutter user interface down to the localized machine learning and storage components isolate tasks to ensure a modular and maintainable application.

4.1.6 System Architecture Diagram

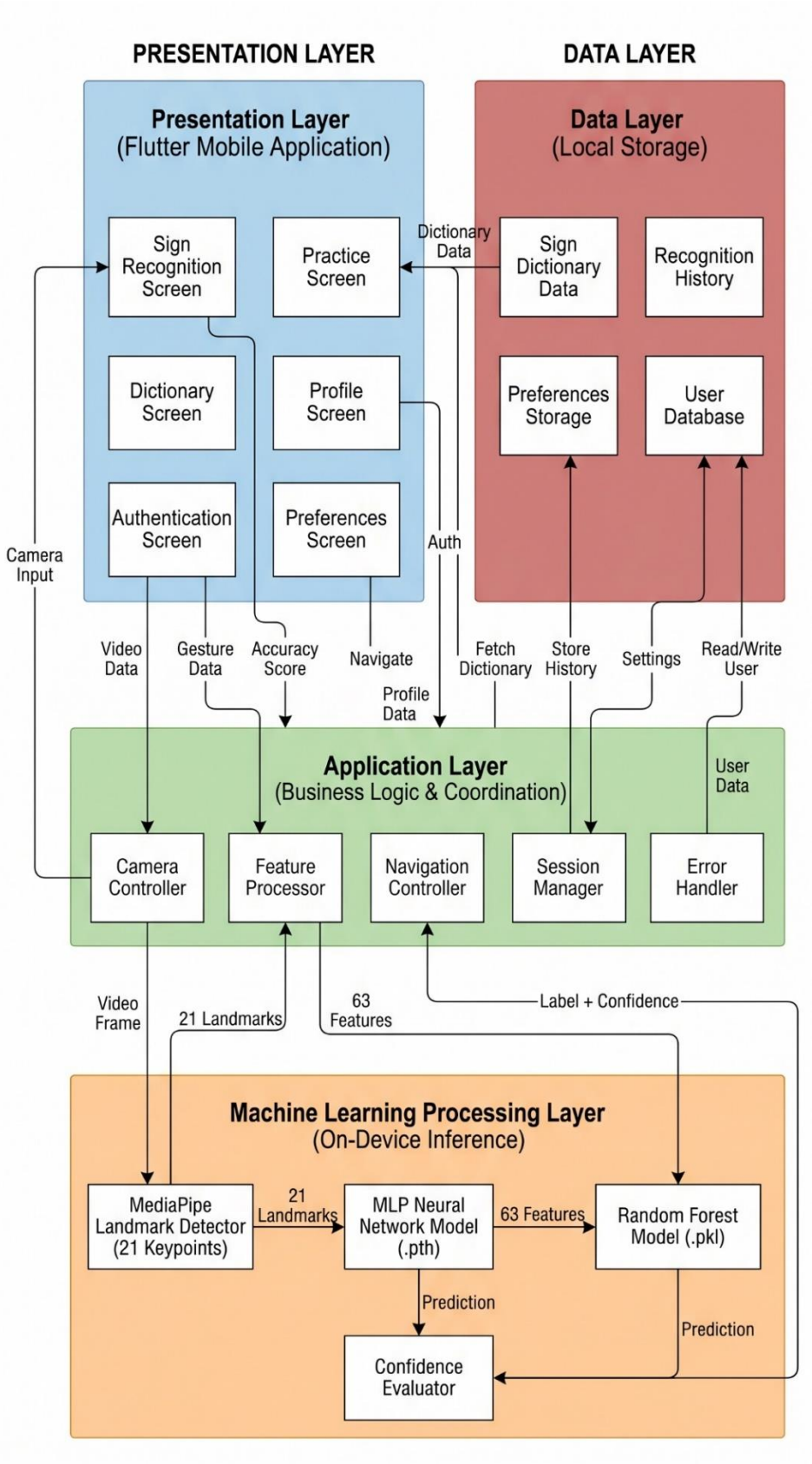

Figure 4-1 System Architecture Diagram

4.2 Data Representation

Data representation describes how information is structured, stored, and related within the system. In the Sign Language Translator, proper data representation is essential because multiple types of information must be organized and connected in a meaningful way. These include user information, gesture feature records, classification results, and dictionary entries.

The data design of the system is centered around the following main entities:

4.2.1 User Entity

The User entity is the most fundamental data element because all personalized features of the system depend on it. A user record stores identifying and authentication-related information. The key attributes of the User entity include user identifier, name, email, password, and preference settings. Through this entity, the system determines who is interacting with the application and retrieves relevant stored information.

4.2.2 Gesture Record Entity

The Gesture Record entity represents the landmark-based feature data extracted during sign recognition. Each gesture record contains a 63-dimensional feature vector derived from 21 hand landmarks. The key attributes include a record identifier, the feature values, the predicted label, and the confidence score. This entity is used during training, evaluation, and real-time inference.

4.2.3 Sign Dictionary Entity

The Sign Dictionary entity contains predefined sign entries that the system is trained to recognize. Each entry includes a sign identifier, sign label, description, and optional image or animation reference. This entity supports the dictionary and learning module of the application.

4.2.4 Preferences Entity

The Preferences entity stores user-specific application settings. Key attributes include audio output preference, theme selection, language preference, and confidence threshold setting. This entity is linked to the User entity and is retrieved when the user logs into the system.

4.2.5 Data Representation Summary

The relationship among entities can be described as follows. One User may have one set of Preferences. One User may generate many Gesture Records during recognition sessions. Each Gesture Record corresponds to one Sign Dictionary entry based on the predicted label. This entity relationship ensures that the system maintains organized and retrievable information.

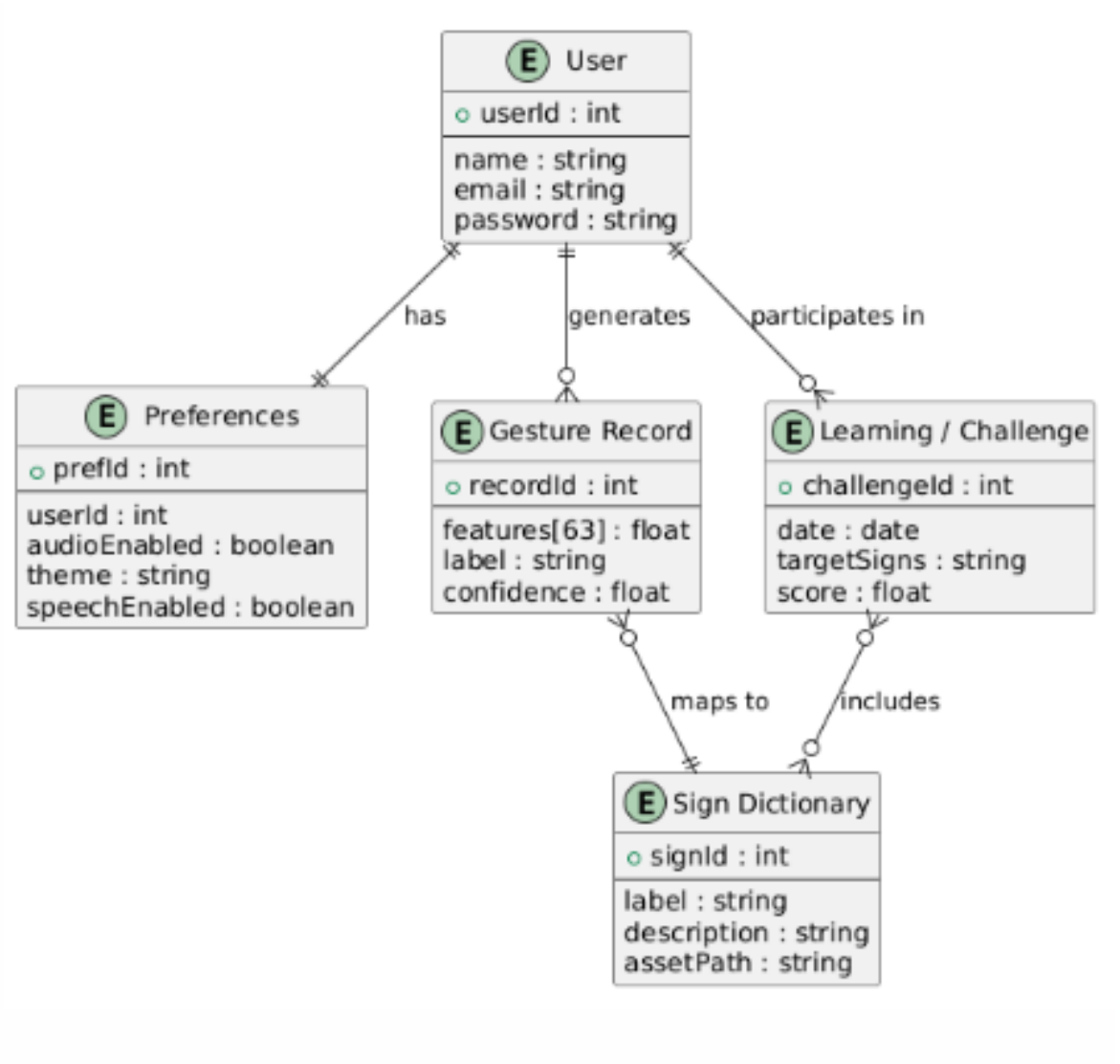

Figure 4-2 Entity Relationship Diagram

Table 4.2 Data Entities and Attributes

Entity	Key Attributes	Description
User	User Id, name, email, password	Stores user account information
Gesture Record	Record Id, features [63], label, confidence	Stores feature data and predictions
Sign Dictionary	Sign Id, label, description	Stores supported sign entries
Preferences	Pref Id, user Id, audio Enabled, theme	Stores user-specific settings

Learning	Challenge Id, date, user Id (optional for tracking), score / accuracy	daily randomized challenges, progress

Table 4.2 shows the entity specification table outlines the complete data model and relational schema designed to support the system's storage requirements. It identifies the primary data structures, tracking attributes, and object relationships necessary to maintain users, gesture logs, dictionary files, and learning analytics

4.3 Process Flow / Representation

Process flow representation explains how the Sign Language Translator performs its major operations step by step during execution. While the architecture section describes the structural components, the process flow explains how these components cooperate when a user performs an action.

4.3.1 Sign Recognition Process Flow

The most important process in the system is the sign recognition flow. This process begins when the user opens the sign recognition screen on the mobile application. The application initializes the camera and begins capturing video frames. Each frame is passed to the Media Pipe landmark detector, which identifies 21 hand key points and returns their x, y, and z coordinates.

The 63 coordinate values are organized into a structured feature vector by the feature processor. This vector is then passed to the loaded machine learning model (Random Forest or MLP) for classification. The model performs multiclass classification and returns a predicted sign label along with a confidence score. If the confidence score meets the defined threshold, the predicted label is displayed on the screen. If audio output is enabled, the user may press the Play button to hear the recognized sign.

The sign recognition process can be described through the following stages:

- User opens recognition screen
- Camera initializes and begins streaming
- Frame captured
- Media Pipe detects 21 hand landmarks
- 63-dimensional feature vector generated
- Feature vector passed to ML model

- Model predicts sign label
- Confidence check performed
- Predicted label displayed
- Optional audio output generated

4.3.2 Authentication and Profile Management Flow

The authentication flow begins when the user launches the application. The system presents options to log in, register, or continue as a guest. If the user selects login, credentials are validated and a session is created. Authenticated users gain access to additional features such as profile management and preferences.

The profile management flow begins when an authenticated user navigates to the profile section. The system retrieves stored profile data and displays it on the screen. The user may edit fields and submit changes. The system validates the input and saves the updated information.

4.3.3 Practice Module Flow

The practice module flow begins when the user selects a sign to practice. The system displays a reference description of the target sign. The user performs the gesture in front of the camera. The system extracts landmarks, generates a feature vector, and passes it to the ML model. The predicted label is compared with the target sign and an accuracy score is displayed to the user.

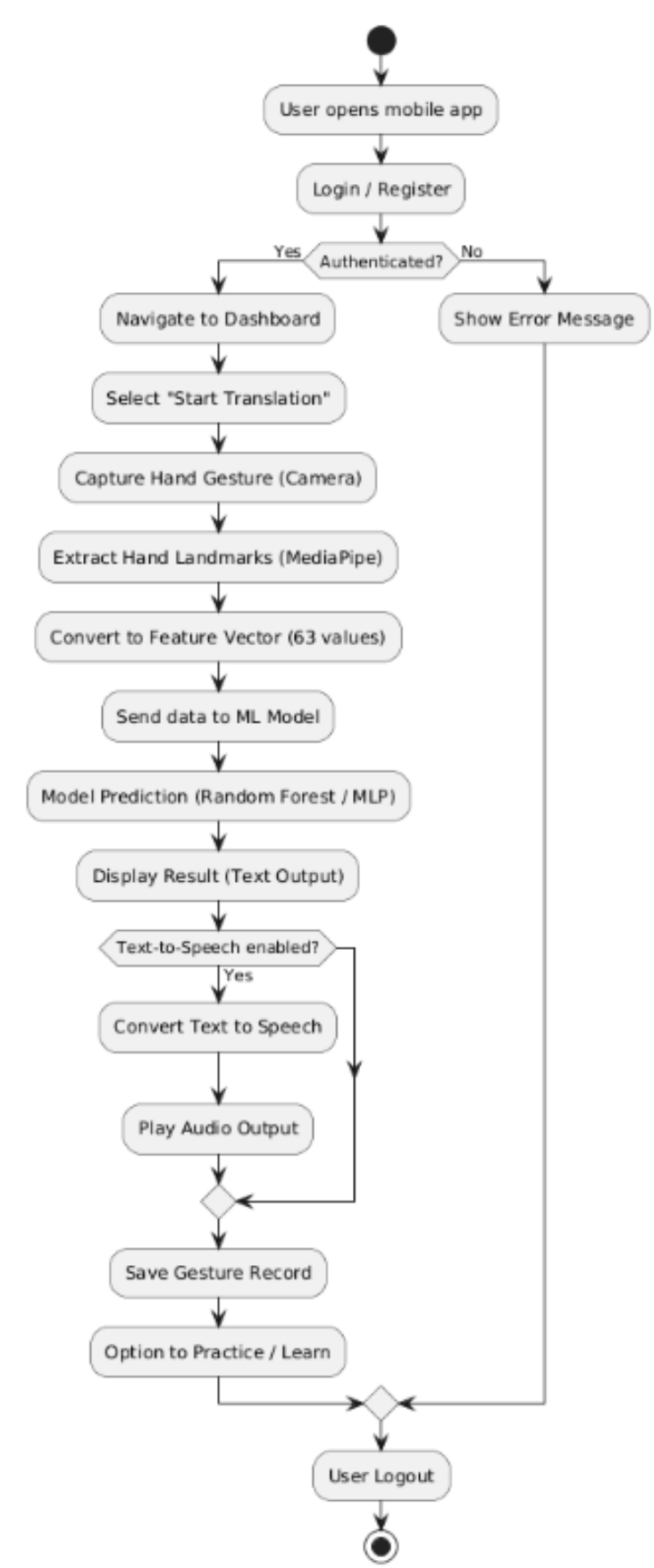

Figure 4-3 System Process Flow Diagram

4.4 Design Models

Design models provide a formal representation of the system structure and behavior. They help convert the functional requirements and architectural decisions into clearly understandable visual representations. The Sign Language Translator uses the following design models.

4.4.1 Sequence Diagram

The sequence diagram illustrates the chronological interaction between system components during sign recognition and profile management. The process begins when the user activates the recognition mode. The mobile application captures camera frames and sends them to the MediaPipe module for landmark detection. Twenty-one landmarks are detected and converted into a 63-dimensional feature vector. These features are passed to the trained machine learning model (Random Forest [6] or MLP), which performs classification and returns a predicted sign label along with confidence score. The recognized text is displayed to the user, and optional audio output is generated if requested.

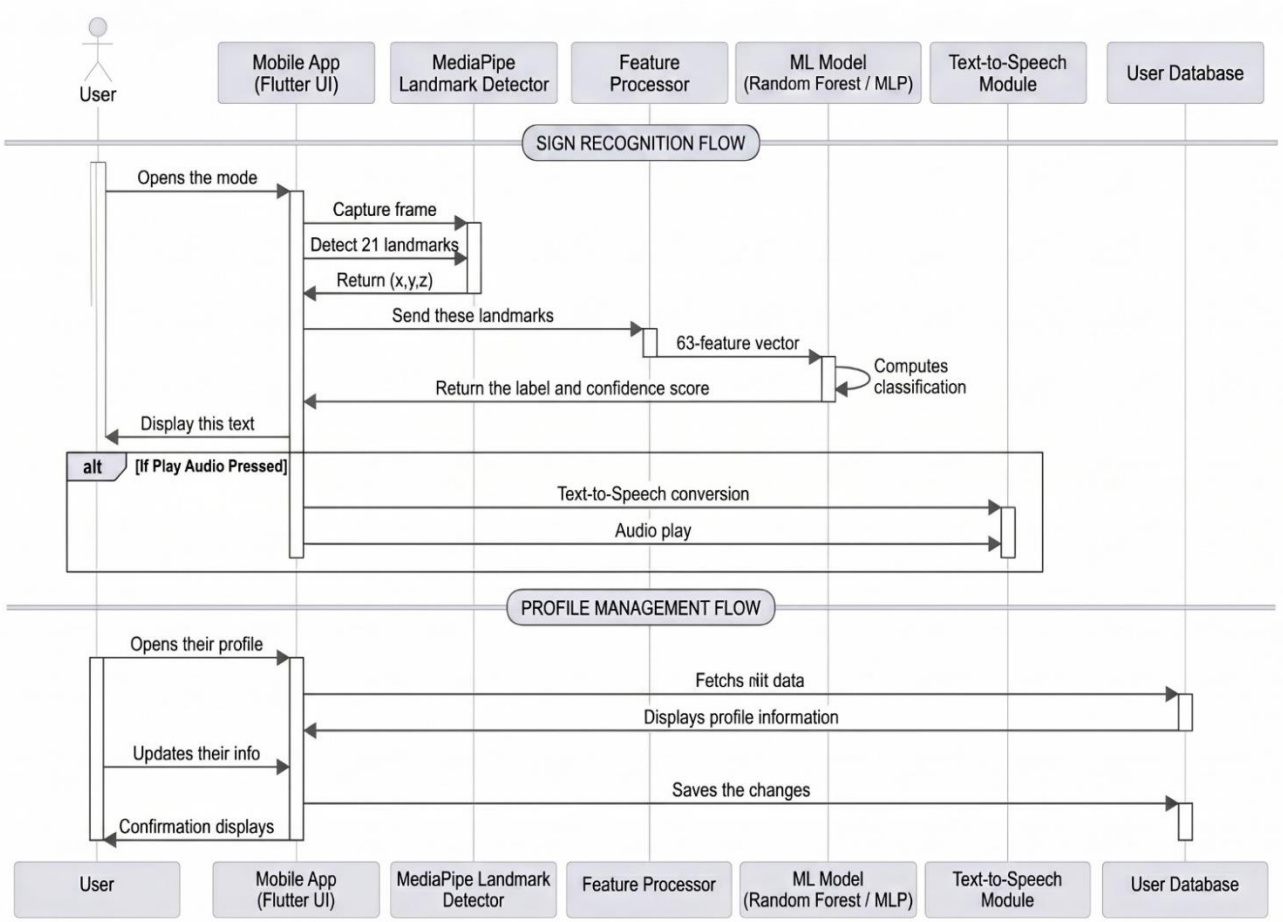

Figure 4-4 Sequence Diagram

4.4.2 Class Diagram

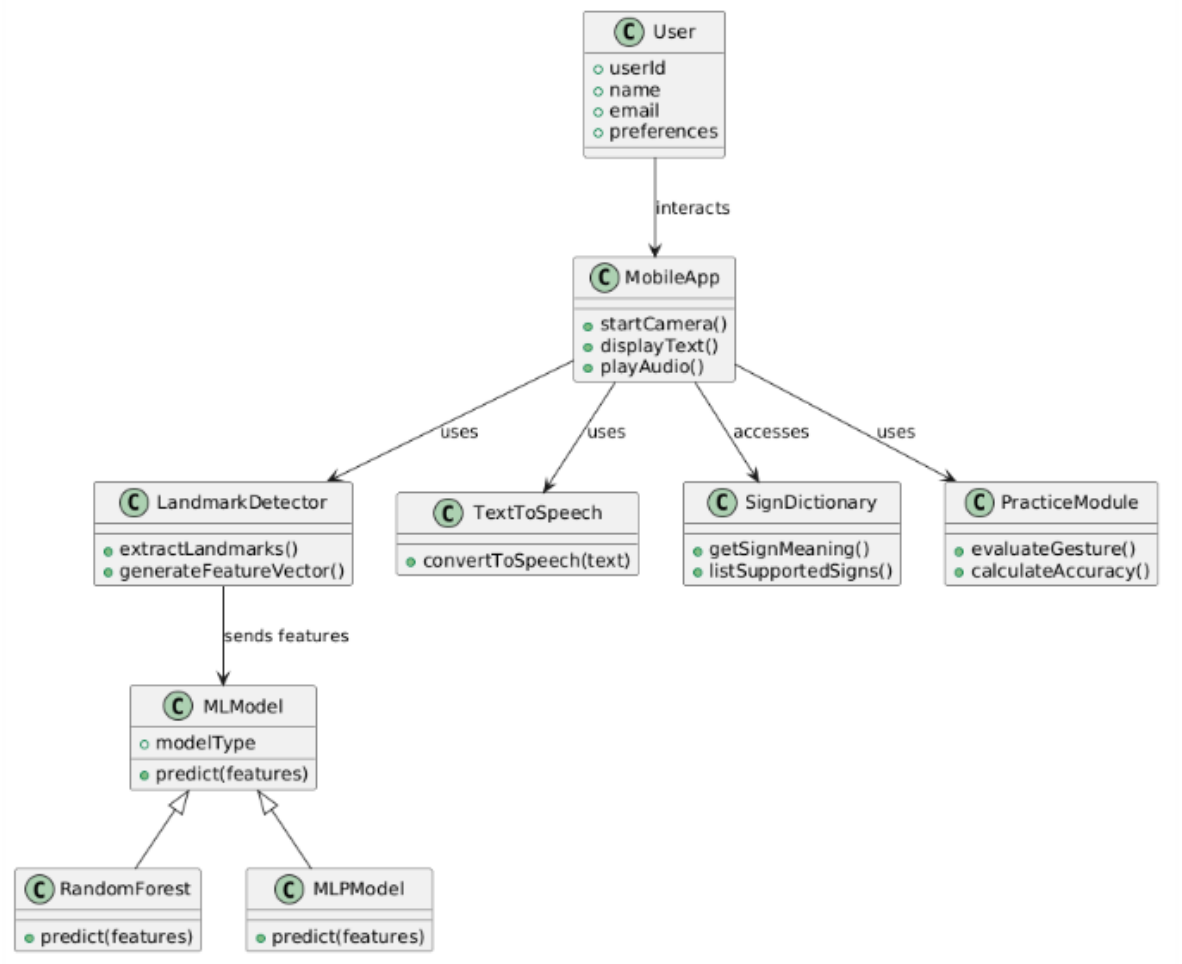

Figure 4-5 Class Diagram

The class diagram represents the structural organization of the Sign Language Translator system. The User class stores user-related information and supports profile updates. The Mobile App class acts as the main controller of user interaction and coordinates system components. The Landmark Detector class is responsible for extracting 21 hand landmarks using Media Pipe. The Feature Processor converts these landmarks into a structured 63-dimensional feature vector.

The ML Model class serves as a parent abstraction for the classification models, with Random Forest Model and MLP Model inheriting its behavior. This design allows flexibility in switching between different trained models. The Text to Speech module provides optional audio output functionality. The User Database manages persistent storage of user data. Additional modules such as Sign Dictionary and Practice Module support learning and feedback functionalities.

Chapter 5
Implementation

5 Implementation

The implementation phase of the AI Sign Language Translator project represents a critical stage in the overall system development lifecycle, where the conceptual design and architectural planning are transformed into a fully functional and operational software solution. This phase focuses on the practical realization of all previously defined requirements, including functional features, system workflows, and integration of intelligent components. The primary objective of the implementation stage is to ensure that the designed system is translated into a reliable, efficient, and user-friendly application capable of addressing real-world communication challenges faced by deaf and mute individuals.

During this phase, multiple system [7] components are developed and integrated, including the mobile application interface, gesture recognition pipeline, and machine learning-based prediction engine. The implementation ensures that these components work cohesively as a unified platform capable of performing real-time sign language translation. Special attention is given to maintaining a balance between computational efficiency and prediction accuracy so that the system can operate smoothly on standard mobile devices without requiring high-end hardware resources.

The system follows a modular implementation approach, which plays a significant role in simplifying development and enhancing maintainability. In this approach, each major component of the system such as the user interface, gesture detection module, and machine learning model is developed as an independent unit. This allows for focused development, easier debugging, and effective testing of individual modules before integration. Once each module is verified to function correctly, they are combined into a complete system, ensuring seamless interaction and data flow between different layers of the application. This modular structure also supports future scalability, as new features or improvements can be added without significantly affecting the existing system.

The mobile application is developed using the Flutter framework, which provides a flexible and efficient environment for building cross-platform applications with a consistent user interface. Flutter enables the implementation of responsive design elements, smooth navigation, and real-time interaction with device hardware, particularly the camera, which is essential for capturing hand gestures. The use of Flutter ensures that the application remains lightweight while delivering a high-quality user experience.

On the other hand, the gesture recognition and machine learning components are implemented using Python-based tools and libraries, which are widely used for artificial intelligence and data processing tasks. The gesture recognition pipeline utilizes MediaPipe for detecting hand landmarks in real time, while the machine learning models are developed using libraries such as Scikit-learn and [8] PyTorch. These tools provide robust support for feature extraction, model training, and prediction, enabling the system to accurately classify hand gestures based on extracted landmark data.

5.1 System Implementation Overview

The system is implemented by dividing it into three major components:

- Mobile Application (Frontend)
- Gesture Recognition Module [9]
- Machine Learning Model

The Flutter-based mobile application provides the user interface and handles user interaction, including gesture capture using the device camera. The gesture recognition module processes the captured frames and extracts hand landmarks using MediaPipe. These extracted features are then passed to the trained machine learning models for classification.

The first component, the Mobile Application (Frontend), is developed using the Flutter framework and serves as the primary interface through which users interact with the system. It is responsible for handling all user-facing functionalities, including navigation, interaction, and real-time display of results. One of the most critical roles of the frontend is to manage gesture capture through the device camera. The application utilizes camera integration to continuously capture frames of the user's hand gestures in real time. Additionally, the frontend ensures a smooth and responsive user experience by managing asynchronous operations, updating the interface dynamically, and presenting prediction results in a clear and readable format. The design of the mobile application emphasizes usability and accessibility, particularly for users who rely on visual feedback for communication.

The second component, the Gesture Recognition Module, acts as the core processing layer that bridges the gap between raw visual input and structured data suitable for machine learning. Once the mobile application captures the gesture frames, they are passed to this module for further processing. The system employs the MediaPipe framework to detect and track hand landmarks within each frame. MediaPipe identifies 21 key points on the hand, each represented

by three-dimensional coordinates (x, y, z), resulting in a total of 63 numerical values per gesture. These landmarks effectively capture the spatial structure and orientation of the hand, providing a compact yet informative representation of the gesture. The module also ensures that the extracted data is properly formatted and normalized before being forwarded to the classification stage. This approach significantly reduces computational overhead compared to traditional image-based processing methods, making it highly suitable for real-time mobile applications.

The third component, the Machine Learning Model, is responsible for interpreting the extracted features and generating meaningful predictions. The system utilizes supervised learning techniques to classify gestures based on the landmark feature vectors. The trained models, including the Random Forest classifier and the Multi-Layer Perceptron (MLP), are designed to recognize predefined gesture patterns and map them to corresponding textual outputs. These models are trained on labeled datasets and optimized to achieve a balance between accuracy and efficiency. During runtime, the processed feature vector is passed to the selected model, which performs classification and returns the predicted label along with confidence measures. This output is then sent back to the mobile application for display.

The integration of these three components forms a seamless and efficient processing pipeline. The workflow begins with gesture capture through the mobile interface, followed by landmark extraction in the gesture recognition module, and finally classification in the machine learning model. The result is then presented to the user in the form of readable text, and optionally converted into speech for enhanced communication. This end-to-end integration ensures that the system operates in real time, providing immediate feedback and enabling natural interaction.

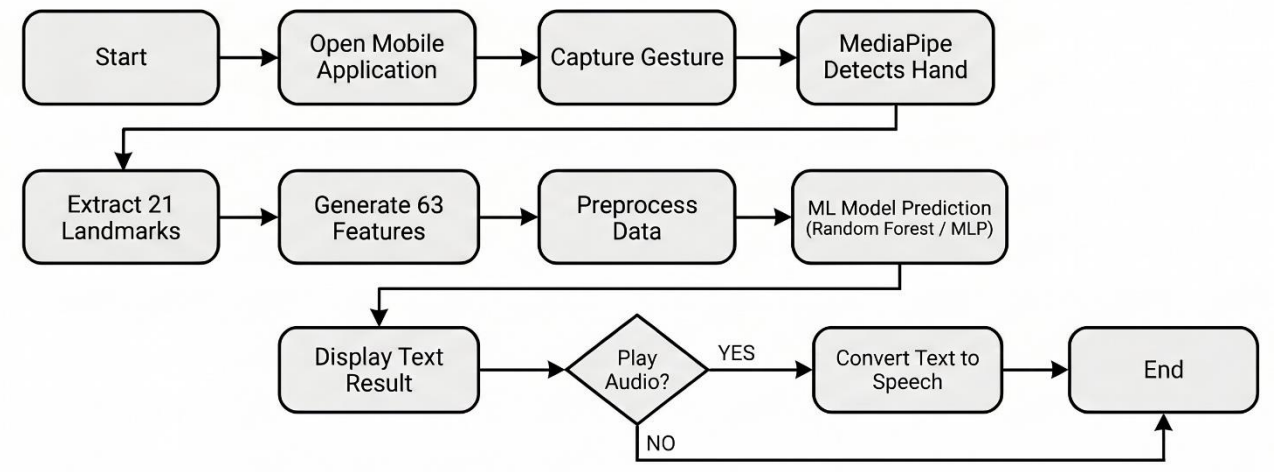

Figure 5-1 Implementation Overview

5.2 Gesture Recognition Implementation

The gesture recognition process in the AI Sign Language Translator system is implemented using the MediaPipe framework, which is a highly efficient and widely adopted solution for real-time hand tracking and landmark detection. MediaPipe provides a pre-trained pipeline that is capable of detecting and tracking hand movements from live video streams with high accuracy and low latency. This makes it particularly suitable for mobile-based applications where real-time performance and computational efficiency are critical requirements.

When the user performs a hand gesture in front of the device camera, the mobile application continuously captures video frames in real time. These frames are then passed to the MediaPipe processing pipeline, which analyzes each frame to detect the presence of a hand and identify its structural features. The framework uses advanced computer vision techniques to isolate the hand region from the background and track its movement across successive frames. This ensures consistent detection even when the hand is in motion or partially occluded.

Media Pipe detects a total of 21 distinct hand landmarks, which correspond to key anatomical points on the hand, including fingertips, joints, and the wrist. Each landmark is represented by three-dimensional coordinates (x, y, z), where the x and y values denote the position of the landmark in the image plane, and the z value represents the relative depth information. These coordinates collectively describe the geometric structure and orientation of the hand in space. By capturing these spatial relationships, the system is able to distinguish between different gestures based on the relative positions of fingers and joints.

The set of 21 landmarks is then transformed into a structured feature vector consisting of 63 numerical values (21 landmarks × 3 coordinates). This feature vector serves as a compact representation of the gesture, effectively summarizing the hand's configuration in a format that can be processed by machine learning algorithms. Compared to raw image data, this representation significantly reduces the dimensionality of the input while preserving the most relevant information required for gesture classification.

Before being passed to the machine learning model, the extracted feature vector undergoes a preprocessing stage to ensure consistency and reliability. This preprocessing may include normalization of coordinate values, scaling, and formatting to match the input requirements of the trained models. Such preprocessing steps help improve model performance by reducing

variations caused by differences in hand size, position, or camera angle. Additionally, this stage ensures that the data remains clean and structured, which is essential for accurate prediction.

One of the key advantages of this approach is that it avoids the need for computationally expensive image-based deep learning models such as Convolutional Neural Networks (CNNs). Instead of processing entire images, the system focuses only on essential hand features, which drastically reduces processing time and resource consumption. This makes the solution highly efficient and well-suited for deployment on mobile devices with limited computational capabilities.

5.3 Machine Learning Model Implementation

The AI Sign Language Translator system utilizes supervised machine learning techniques to perform the classification of hand gestures based on the extracted landmark features. The gesture recognition problem is formulated as a multiclass classification task, where each input feature vector corresponds to a specific sign label. The system relies on labeled training data to learn patterns and relationships between hand landmark configurations and their associated meanings. This supervised learning approach enables the model to generalize from known examples and accurately predict unseen gestures during real-time operation.

To ensure robustness and comparative evaluation, two different machine learning models are implemented and analyzed within the system:

- Random Forest Classifier [10]
- Multi-Layer Perceptron (MLP) [11]

The use of multiple models allows for performance comparison in terms of accuracy, computational efficiency, and suitability for mobile deployment. This approach also provides flexibility in selecting the most appropriate model based on system constraints and real-time requirements.

The Random Forest Classifier is implemented using the Scikit-learn library, which provides a reliable and efficient framework for traditional machine learning algorithms. Random Forest is an ensemble learning method that constructs multiple decision trees during training and combines their outputs to produce a final prediction. Each tree in the forest is trained on a subset of the data, and the final classification is determined through majority voting. This approach helps reduce overfitting and improves generalization performance. Due to its

relatively low computational cost and strong performance on structured data, Random Forest is well-suited for handling the 63-dimensional landmark feature vectors used in this project.

The Multi-Layer Perceptron (MLP) model is implemented using the PyTorch framework, which is widely used for building and training neural networks [12]. The MLP is a feedforward artificial neural network consisting of an input layer, one or more hidden layers, and an output layer. It is capable of learning complex, non-linear relationships between input features and output classes. In this system, the MLP model processes the 63-dimensional feature vector and applies a series of transformations through hidden layers using activation functions. This enables the model to capture subtle variations in hand gestures that may not be easily distinguishable using simpler algorithms. Although the MLP offers greater representational power, it typically requires more computational resources compared to traditional models.

Both models are trained using a labeled dataset consisting of gesture feature vectors and their corresponding sign labels. The dataset is prepared by extracting hand landmarks from multiple gesture samples and assigning appropriate labels to each instance. During the training phase, the dataset is typically divided into training and testing subsets to evaluate model performance. Various techniques such as normalization, data cleaning, and feature consistency checks are applied to ensure the quality of the input data. Model evaluation metrics, such as accuracy, are used to assess performance and select the most suitable model for deployment.

After the training process is completed, the models are saved in serialized formats for later use during inference. The Random Forest model is stored as a .pkl file, which preserves the trained model parameters and structure in a format compatible with Scikit-learn. Similarly, the MLP model is saved as a .pth file, which contains the learned weights and architecture of the neural network as defined in PyTorch. These serialized files allow the models to be loaded directly into the system without requiring retraining.

During runtime, the mobile application interacts with the prediction module to perform real-time gesture classification. Once the gesture recognition module extracts the feature vector from the captured hand landmarks, this data is forwarded to the prediction engine. The system loads the pre-trained model into memory and uses it to process the incoming feature vector. The model then generates a prediction by identifying the most probable class label corresponding to the input gesture.

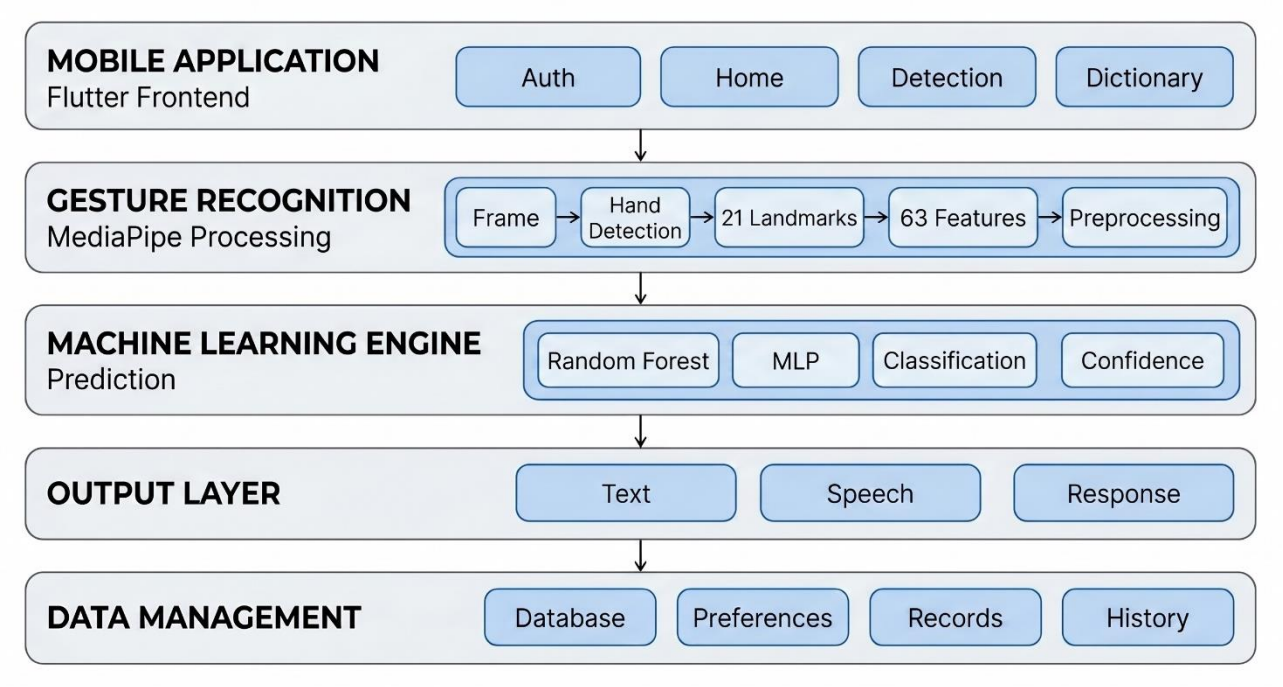

Figure 5-2 Layered System Architecture

5.4 Mobile Application Implementation

The mobile application component of the AI Sign Language Translator system is developed using the Flutter framework, which provides a modern, flexible, and efficient environment for building cross-platform applications. Flutter enables developers to create applications that can run seamlessly on multiple platforms, including Android and iOS, using a single codebase. This significantly reduces development effort while ensuring consistency in design and functionality across different devices. The framework is based on the Dart programming language and offers a rich set of pre-built widgets that facilitate the development of responsive and visually appealing user interfaces.

The main features implemented in the mobile application include:

- User authentication [13] (login and registration)
- Real-time gesture capture using the camera
- Display of predicted text output
- Optional text-to-speech functionality
- Access to sign dictionary
- Practice and learning modules

One of the key advantages of using Flutter is its ability to provide efficient UI rendering through its high-performance rendering engine. This ensures that the application maintains smooth transitions, fast response times, and an overall fluid user experience. These characteristics are particularly important for applications that involve real-time processing, such as gesture recognition, where delays or lag can negatively impact usability.

The mobile application is designed with a multi-screen architecture to support various functionalities and ensure intuitive navigation for users. Each screen is dedicated to a specific task, allowing users to interact with the system in a structured and organized manner. The design emphasizes simplicity and accessibility, keeping in mind the target users, including individuals with hearing and speech impairments. Clear visual elements, readable text, and minimal navigation complexity are incorporated to enhance usability.

5.4.1 User Authentication

The main features implemented in the mobile application include user authentication, which allows users to securely register and log into the system [14]. This feature ensures personalized access and enables the system to manage user-specific data, such as preferences and activity records. The authentication mechanism is designed to be straightforward while maintaining basic security standards.

5.4.2 Real-time Gesture Capture

Another core feature is real-time gesture capture, which utilizes the device camera to continuously capture hand movements. The application integrates camera functionality to stream live video frames, which are then processed by the gesture recognition module. This feature is central to the system, as it enables users to perform sign gestures naturally and receive immediate feedback.

5.4.3 Predicted Text Output

The application also provides a display of predicted text output, where the recognized gesture is converted into readable text and shown on the screen. This output is presented clearly to ensure easy understanding. In addition to text output, the system supports an optional text-to-speech functionality, which converts the predicted text into audio. This feature enhances communication by allowing the translated message to be spoken aloud, making it easier for hearing individuals to understand the user's intent.

5.4.4 Sign Language Dictionary

To support learning and usability, the application includes access to a sign language dictionary, which contains predefined gestures along with their meanings. This feature helps users familiarize themselves with supported signs and serves as a reference tool for both beginners and experienced users. Furthermore, the system includes practice and learning modules, which allow users to test their gestures and improve their accuracy. These modules provide an interactive environment where users can engage with the system and enhance their understanding of sign language.

From a technical perspective, the application leverages [4]Flutter's widget-based architecture to build reusable and modular UI components. Widgets such as buttons, text fields, and camera views are combined to create a cohesive interface. Additionally, the application utilizes asynchronous programming techniques to handle real-time operations efficiently. Since tasks such as camera input processing and model prediction require continuous data handling, asynchronous execution ensures that the user interface remains responsive and does not freeze during processing.

5.5 Integration of System Components

The integration process connects the mobile application with the gesture recognition module and machine learning models. When a user performs a gesture, the following sequence occurs:

- The camera captures the gesture frame.
- Media Pipe extracts hand landmarks.
- Landmarks are converted into a feature vector.
- The feature vector is sent to the ML model.
- The model predicts the gesture label.
- The result is displayed on the screen.

The integration process of the AI Sign Language Translator system is a crucial aspect of the implementation phase, as it ensures effective communication and coordination between the mobile application, gesture recognition module, and machine learning models. This integration transforms individually developed components into a unified system capable of performing real-time sign language translation. The primary objective of this process is to establish a seamless data flow across different layers of the system while maintaining efficiency, accuracy, and low latency.

The system follows a structured pipeline in which each component performs a specific function and passes its output to the next stage. This pipeline begins with user interaction through the mobile application and continues through gesture processing, feature extraction, and machine learning-based prediction. The successful integration of these stages ensures that the system operates as a cohesive unit rather than as isolated modules.

When a user performs a hand gesture in front of the device camera, the process is initiated by the mobile application. The application continuously captures video frames in real time using the device's camera interface. These frames serve as the raw input for the gesture recognition process. The captured frames are immediately forwarded to the gesture recognition module for further processing.

Within the gesture recognition module, the MediaPipe framework is used to analyze each frame and detect the presence of a hand. Once a hand is identified, MediaPipe extracts 21 hand landmarks, each represented by three-dimensional coordinates (x, y, z). These landmarks provide a detailed representation of the hand's structure, including the position of fingers, joints, and wrist. The extraction process is optimized for speed and accuracy, ensuring that the system can operate in real time without noticeable delays.

After the landmarks are extracted, they are transformed into a structured feature vector consisting of 63 numerical values. This transformation step converts spatial hand information into a format suitable for machine learning processing. The feature vector may also undergo preprocessing, such as normalization or scaling, to ensure consistency and improve prediction performance. This step is essential for maintaining the reliability of the system across different users and environmental conditions.

The processed feature vector is then passed to the machine learning model, which serves as the prediction engine of the system. The model, which has been previously trained on labeled gesture data, analyzes the input features and determines the most likely gesture label. Depending on the implementation, either the Random Forest classifier or the Multi-Layer Perceptron (MLP) model is used for this task. The prediction process is designed to be computationally efficient, enabling rapid classification of gestures.

This entire sequence of operations occurs within a very short time frame, enabling real-time interaction between the user and the system. The integration ensures that data flows smoothly between components without bottlenecks or delays. Efficient handling of asynchronous operations and optimized data processing play a key role in achieving this level of performance.

5.6 User Interface Implementation

The user interface (UI) of the AI Sign Language Translator system is carefully designed to be simple, intuitive, and highly accessible, with a particular focus on the needs of users with hearing or speech impairments. Since the target audience relies heavily on visual interaction rather than auditory cues, the interface emphasizes clarity, readability, and ease of use. The design ensures that users can navigate the application effortlessly, understand system outputs without confusion, and interact with different features in a smooth and efficient manner.

A user-centered design approach is adopted, where the interface is structured to minimize cognitive load and reduce unnecessary complexity. Clear visual hierarchies, consistent layouts, and responsive elements are incorporated to enhance usability. The application avoids cluttered screens and instead uses well-organized components, readable fonts, and intuitive icons to guide user interaction. Special attention is given to ensuring that all essential functionalities are easily accessible without requiring extensive navigation or prior technical knowledge.

The main UI screens include:

- **Home Screen:** Entry point of the application
- **Login and Signup Screen:** Login User and Register new user
- **Gesture Detection Screen:** Displays camera feed and prediction results
- **Dictionary Screen:** Shows available signs and meanings
- **Practice Screen:** Allows users to test gestures
- **Profile Screen:** Displays user information

The mobile application is structured into multiple UI screens, each dedicated to a specific functionality. This modular screen-based design improves user experience by separating different tasks and providing a clear workflow throughout the application.

5.6.1 Home Screen

The Home Screen serves as the entry point of the application. It provides users with access to the main features of the system through clearly labeled options or buttons. This screen acts as a central hub from which users can navigate to gesture detection, dictionary, practice modules, and profile management. The layout is designed to be simple and welcoming, ensuring that users can quickly understand how to proceed.

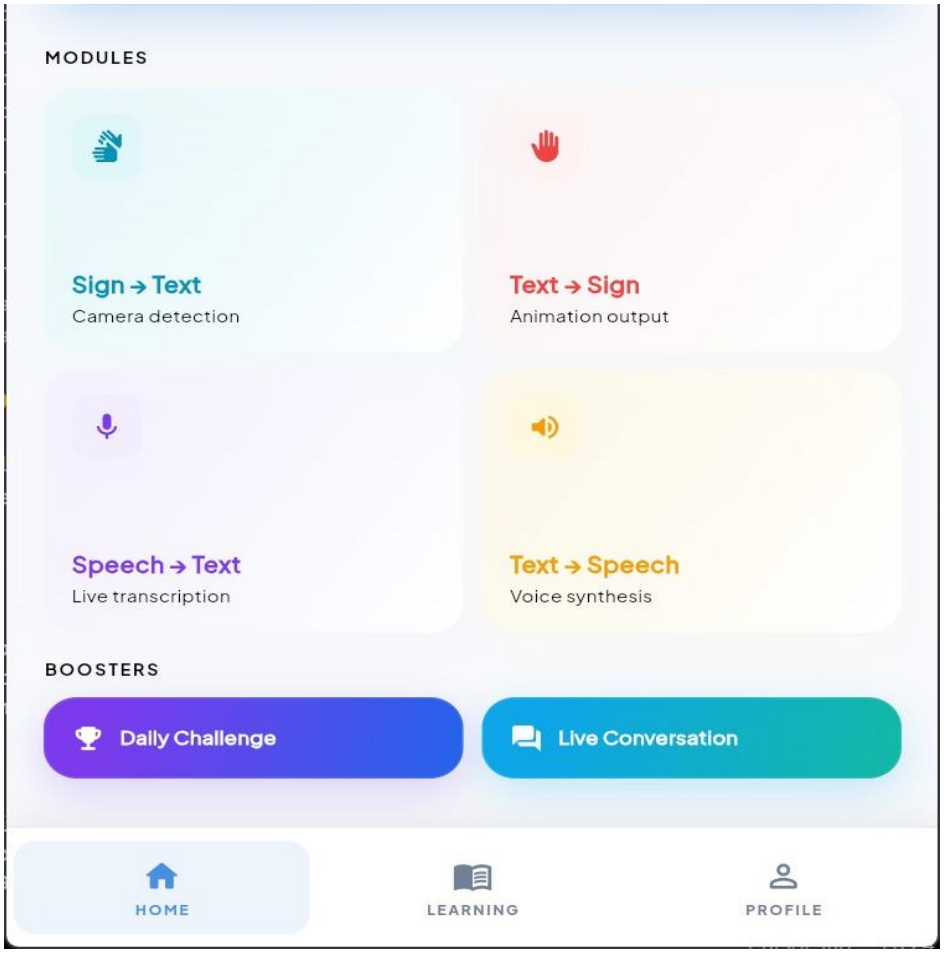

Figure 5-3 Home Screen

5.6.2 Login and Signup Screen

The Login and Registration module of Sign Bridge provides secure and user-friendly access to the application while maintaining a clean and accessible interface design. The Registration screen allows new users to create an account by entering essential credentials and profile information, while the Login screen enables existing users to securely access personalized features such as learning progress, daily challenges, and application preferences. The module is integrated within the Flutter-based architecture using organized UI components and Provider-based state management to ensure smooth navigation and responsive interaction.

Consistent theming through App Theme, App Colors, and App Text Styles enhances readability and accessibility across both light and dark modes, contributing to an inclusive user experience for all users.

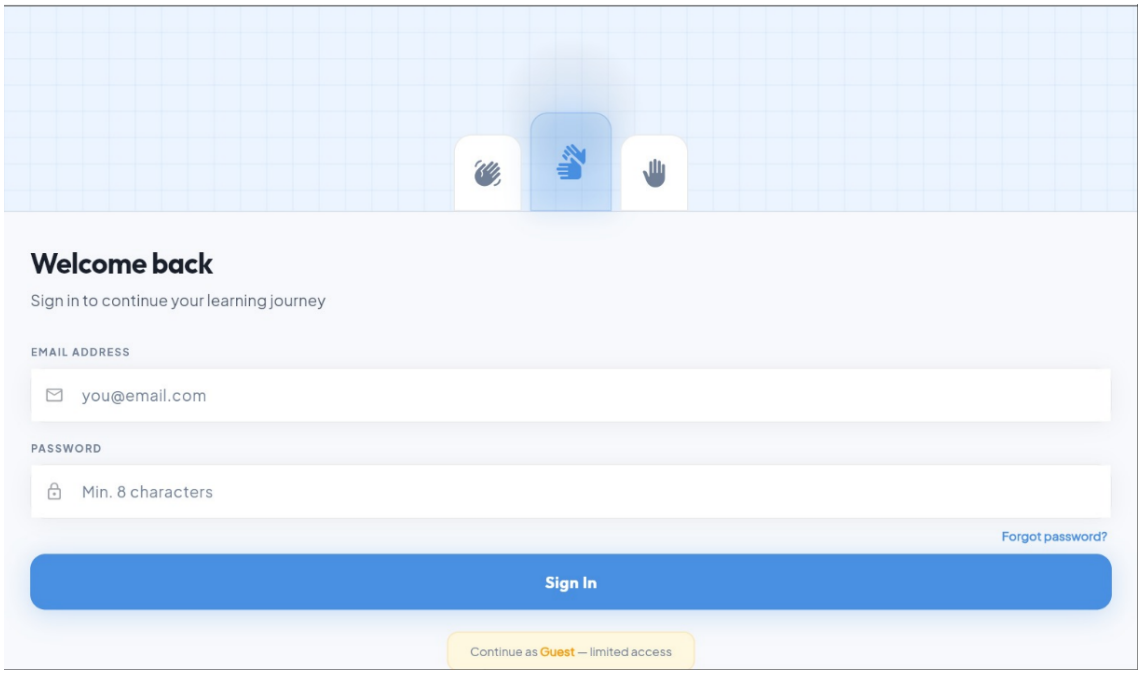

Figure 5-4 Login Screen

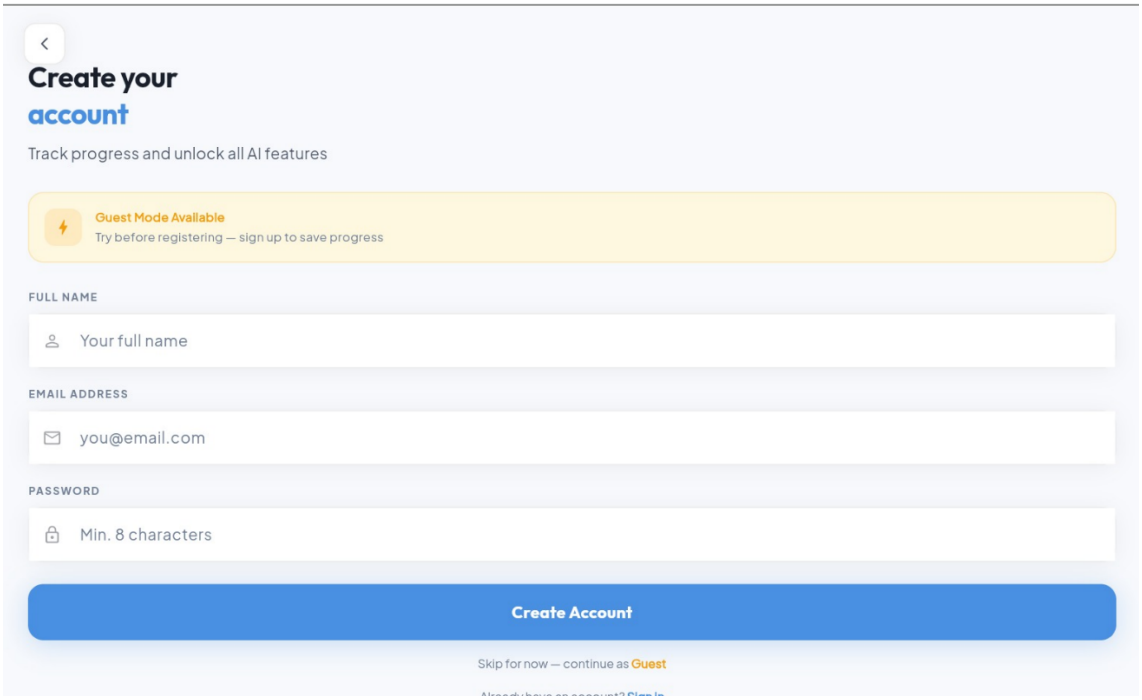

Figure 5-5 Signup Screen

5.6.3 Gesture Detection Screen

The Gesture Detection Screen is one of the most critical components of the interface. It displays the live camera feed and allows users to perform hand gestures in real time. The screen also shows the predicted output, typically in the form of text, which is updated dynamically as the system processes the gesture. Visual indicators may be used to guide the user in positioning their hand correctly within the camera frame.

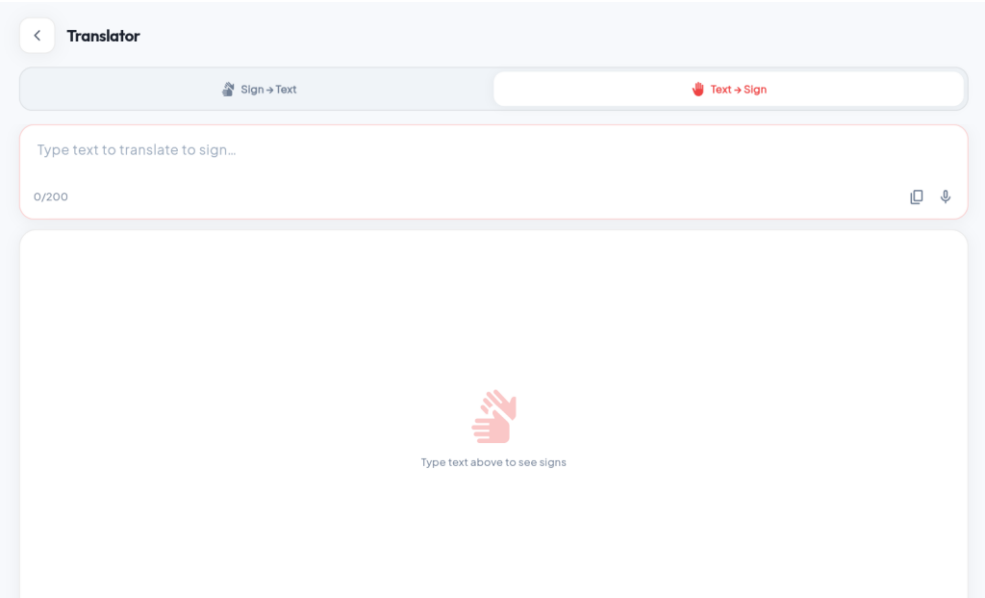

Figure 5-6 Gesture Detection Screen

5.6.4 Learning Screen

The Learning Screen provides users with a collection of supported sign gestures along with their corresponding meanings. This screen serves as an educational and reference tool, helping users understand which gestures are recognized by the system. It may include images or descriptions of gestures to enhance comprehension. The dictionary feature supports both new learners and experienced users by offering a structured way to explore sign language vocabulary.

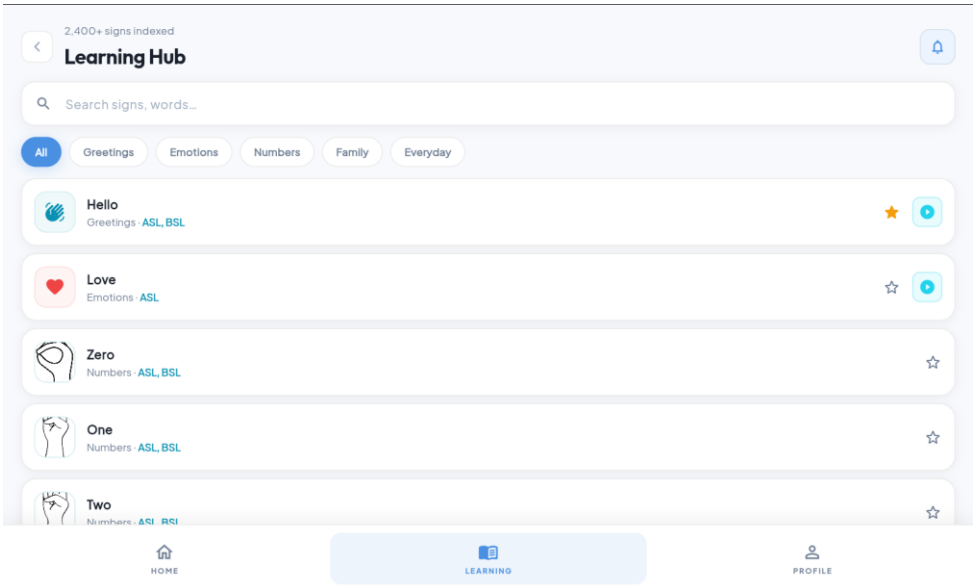

Figure 5-7 Learning Screen

5.6.5 Practice Screen

The Practice Screen is designed to provide an interactive learning environment where users can test their gestures and evaluate their performance. Users can attempt specific signs and receive feedback based on the system's predictions. This feature encourages learning and skill improvement by allowing users to practice repeatedly and observe how accurately their gestures are recognized. It plays an important role in increasing user confidence and familiarity with the system.

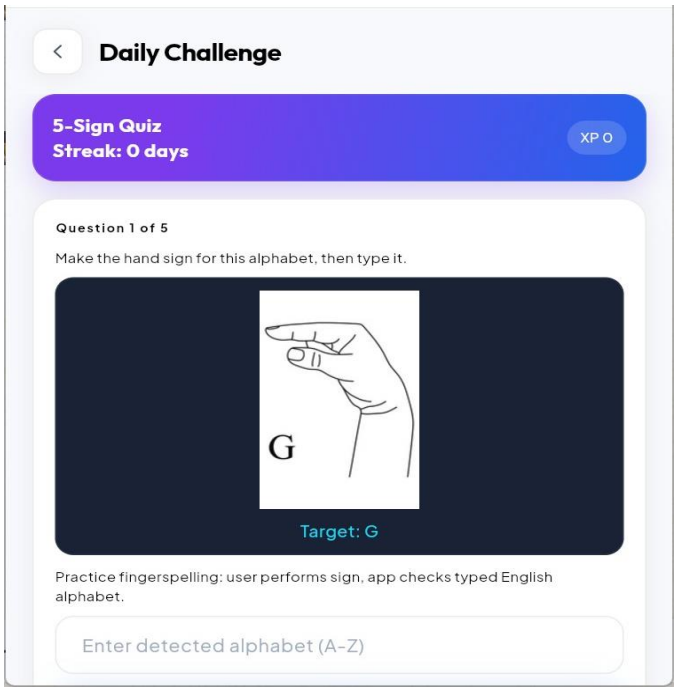

Figure 5-8 Practice Screen

5.6.6 Profile Screen

The Profile Screen allows users to view and manage their personal information, such as name, email, and preferences. It may also include options for updating account details or adjusting application settings. This screen ensures that user-related data is organized and accessible in a dedicated section of the application.

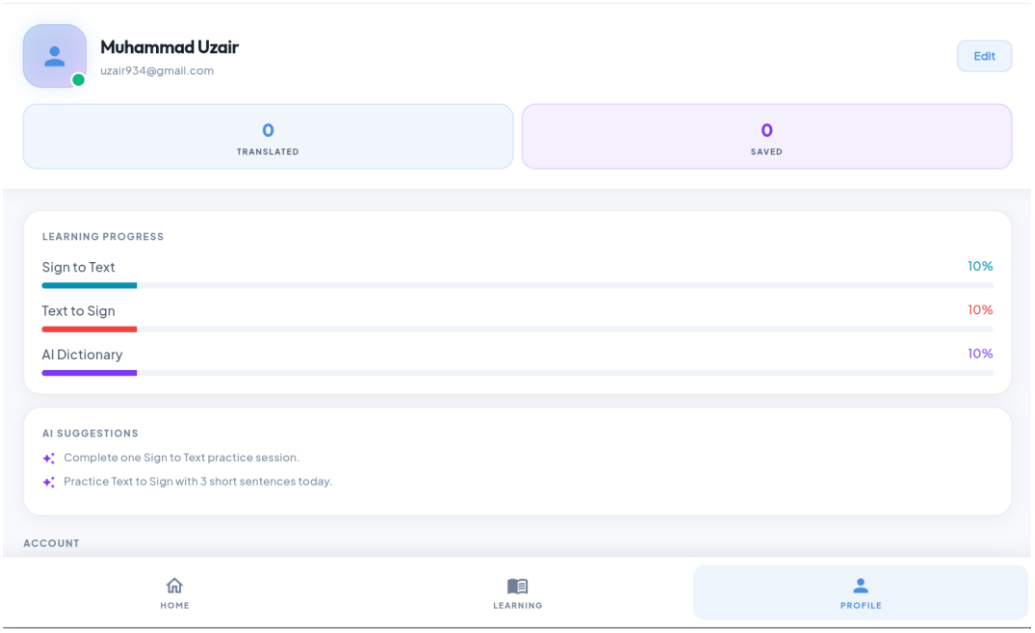

Figure 5-9 Profile Screen

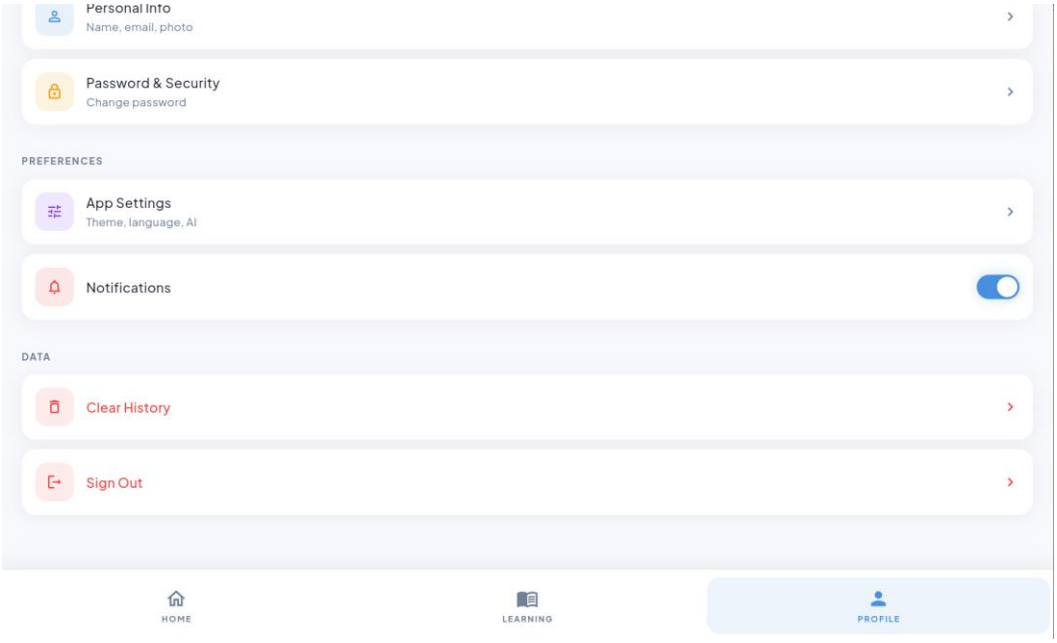

Figure 5-10 Profile Screen 2

5.6.6 Human Computer Interaction (HCI)

The overall UI design follows established Human Computer Interaction (HCI) [15] principles, including usability, accessibility, consistency, and feedback. The system ensures that users receive immediate visual feedback for their actions, which is essential for maintaining engagement and understanding system responses. Navigation is kept consistent across all screens, and interactive elements are designed to be responsive and easy to use.

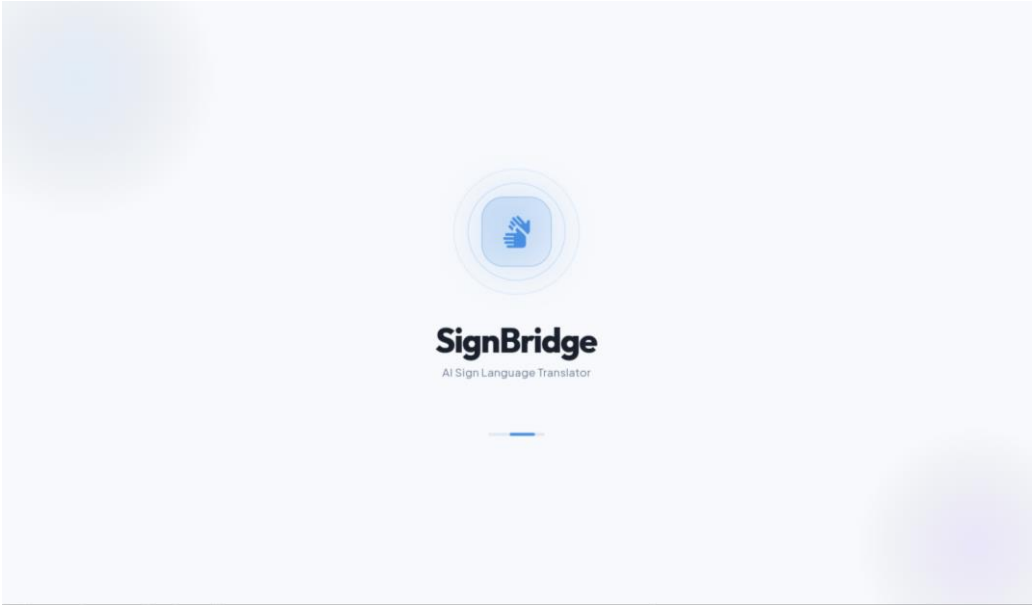

Figure 5-11 HCI

Additionally, accessibility considerations are integrated into the design, such as clear text display, minimal reliance on audio cues, and intuitive visual indicators. These design choices ensure that the application remains inclusive and effective for its intended user base.

5.7 Data Handling and Storage

The AI Sign Language Translator system incorporates a structured and efficient data handling mechanism to manage various types of information generated and utilized during system operation. The primary categories of data handled by the system include gesture records, user information, and model output data. Proper management of this data is essential to ensure system reliability, performance consistency, and a secure user experience

Gesture-related data forms a significant portion of the system's operational information. Each time a user performs a gesture, the system processes it and generates a corresponding prediction. These interactions can be recorded as gesture records, which typically include the predicted gesture label, the time and date of the interaction (timestamp), and the associated

confidence score generated by the machine learning model. In some cases, additional metadata such as user ID or session details may also be included. Maintaining such records allows the system to track user activity, analyze performance, and support features such as practice evaluation and progress monitoring.

The system also manages user-related data, which includes information such as user credentials, profile details, and application preferences. This data is essential for enabling personalized interaction with the application. For example, user authentication ensures that only registered users can access certain features, while stored preferences can be used to customize the user experience. The design ensures that user data is logically organized and easily retrievable when needed, supporting smooth application functionality.

In addition to gesture and user data, the system handles model output data, which includes the results generated by the machine learning models. This data primarily consists of predicted labels and confidence scores that indicate the reliability of each prediction. While some of this information is used temporarily for real-time display, certain outputs may be stored for future reference, analysis, or improvement of system performance.

To manage this data effectively, the system implements structured data handling techniques that ensure organized storage and efficient retrieval. Data is maintained in a format that allows quick access and minimal processing delay, which is particularly important for real-time applications. The system design supports scalability, allowing additional data fields or records to be incorporated without affecting existing functionality.

Security is a key consideration in the data handling process. The system ensures that user-related data is securely managed and protected from unauthorized access. Authentication mechanisms are implemented to restrict access to user-specific information, ensuring that only authorized users can view or modify their data. Basic data protection practices, such as controlled access and secure storage, are followed to maintain user privacy and system integrity.

Furthermore, the system is designed to handle data efficiently during runtime without causing performance degradation. Since the application operates in real time, data processing and storage operations are optimized to avoid delays or interruptions. Temporary data generated during gesture recognition is handled in memory, while important records are stored in a structured manner for long-term use.

Chapter 6
Testing and Evaluation

6 Testing and Evaluation

This chapter presents a comprehensive and systematic evaluation of the AI Sign Language Translator application, focusing on the verification and validation of the system developed during the implementation phase. The evaluation process is essential to ensure that the system operates as intended and delivers reliable performance under various conditions. This chapter provides a detailed overview of the testing strategies employed, the design and execution of test cases, and the results obtained during different phases of testing. By thoroughly analyzing the system's behavior, this chapter aims to confirm that the application meets the specified requirements and maintains a high level of accuracy, efficiency, and usability.

The primary objective of this chapter is to assess whether the developed system satisfies both the functional and non-functional requirements defined in the Software Requirements Specification (SRS). Functional requirements, such as gesture recognition, real-time prediction, and user interaction, are tested to ensure that all features operate correctly and consistently. At the same time, non-functional requirements, including performance, usability, responsiveness, and reliability, are evaluated to determine whether the system provides a smooth and effective user experience. This dual focus ensures that the application is not only functionally correct but also practical for real-world use.

Testing plays a critical role in identifying potential issues, inconsistencies, or performance limitations within the system. Therefore, a structured testing approach is adopted to evaluate each component individually as well as the system as a whole. Various testing levels are applied, including unit testing, where individual modules such as the gesture recognition component or prediction engine are tested independently; integration testing, where interactions between components are verified; and system-level testing, where the complete application is evaluated in a real-world environment. This layered testing strategy ensures that errors are detected early and resolved effectively, reducing the risk of failures during actual usage.

The testing process is conducted throughout the development lifecycle by following Agile software development principles. Instead of performing testing only at the final stage, continuous testing is integrated into each development iteration. This approach allows for regular feedback, incremental improvements, and rapid identification of issues. As new features are implemented, corresponding test cases are designed and executed to validate their

functionality. This iterative testing process enhances the overall quality of the system and ensures that each component meets the required standards before moving to the next stage.

6.1 Testing Strategy

A comprehensive multi-level testing approach was adopted to ensure thorough validation of the AI Sign Language Translator system. This approach was designed to evaluate the application at different levels of granularity, from individual components to the complete integrated system. By employing multiple testing layers, the system was examined for correctness, performance, usability, and reliability under a variety of conditions. This structured testing strategy ensured that both the internal logic of the system and the overall user experience were rigorously verified.

- **Unit Testing:** Individual components and functions
- **Integration Testing:** Interaction between modules (Camera + Media Pipe [16] + TF Lite + UI)
- **Functional Testing**: Verification against SRS functional requirements
- **Non-Functional Testing**: Performance, usability, accuracy, and reliability
- **User Acceptance Testing (UAT):** Informal testing with target users

Tools Used:

A variety of tools and techniques were used to support the testing process. The Flutter Test framework was utilized for unit and widget testing, enabling automated validation of UI components and application logic. In addition, extensive manual testing was performed on physical Android devices to simulate real-world usage conditions and identify issues that may not be captured through automated testing alone. The Android Profiler tool was used to analyze system performance, including CPU usage, memory consumption, and frame rendering efficiency, ensuring that the application operates smoothly on resource-constrained devices. Additionally, custom scripts were developed to perform batch evaluation of the machine learning model, allowing systematic testing of prediction accuracy across multiple gesture samples.

- Flutter Test framework for unit and widget testing
- Manual testing on physical Android devices
- Android Profiler for performance analysis
- Custom scripts for batch accuracy evaluation of the ML model

Test Environment:

The testing process was conducted within a well-defined test environment to ensure consistency and reliability of results. Multiple Android devices were used, including models from the Samsung Galaxy A series and Redmi Note series, with hardware configurations ranging from 4GB to 8GB RAM. This variation in device specifications helped evaluate the system's performance across both mid-range and relatively higher-end smartphones. The application was tested on different versions of the Android operating system, ranging from Android 10 to Android 14, ensuring compatibility across a wide range of devices.

- Devices: Samsung Galaxy A series, Redmi Note series (4GB–8GB RAM)
- OS: Android 10 to Android 14
- Lighting conditions: Indoor normal, low light, and outdoor

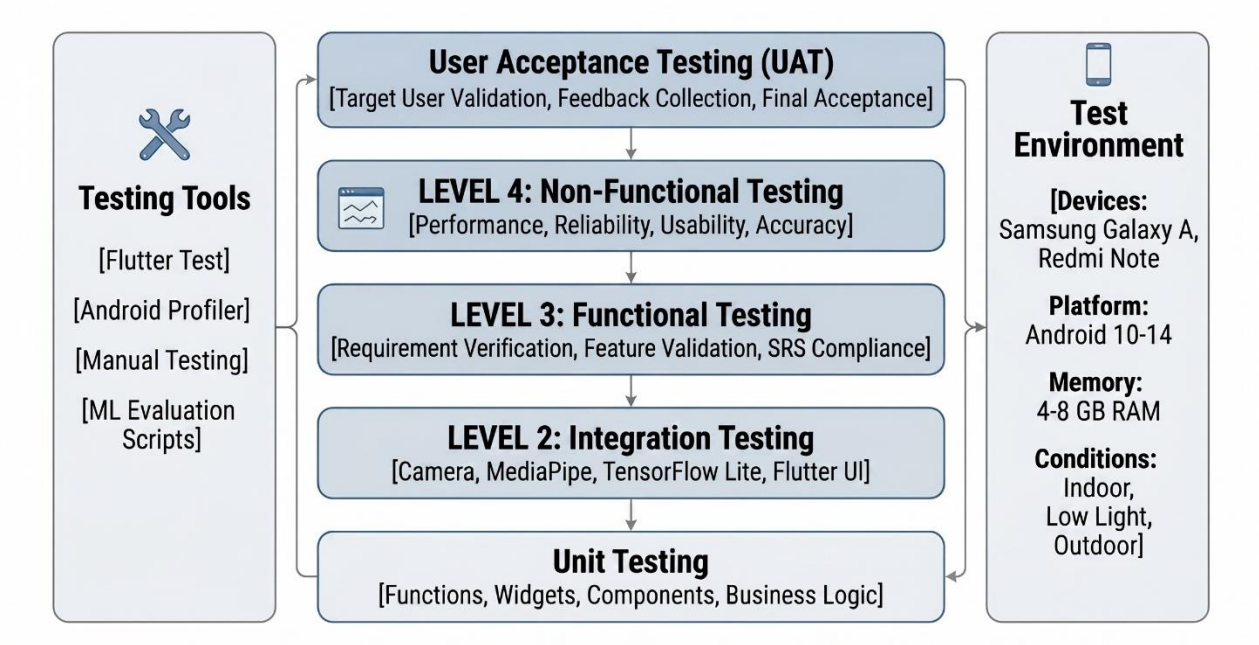

Figure 6-1 Testing Strategy

6.1.1 Unit Testing

At the first level, Unit Testing was performed to validate individual components and functions in isolation. Each module, such as gesture feature extraction, model prediction logic, and UI components, was tested independently to ensure that it performed its intended functionality correctly. Unit testing helped identify low-level errors early in the development process, making debugging more efficient and preventing issues from propagating to higher levels of the system.

Sample Unit Test Cases:

Table 6.1 Unit Test Cases

Test ID	Component	Test Description	Expected Result	Status
UT-01	Landmark Extractor	Extract 63 features from valid hand landmarks	Returns normalized 63D vector	Pass
UT-02	TF Lite Interpreter	Load and run inference on sample input	Returns valid prediction with confidence	Pass
UT-03	Feature Normalizer	Normalize coordinates relative to wrist	"Values in range [0 1]"	Pass
UT-04	TTS Service	Trigger manual audio playback	Audio plays only on button press	Pass

Table 6.1 shows the unit testing matrix outlines the verification criteria, expected execution behaviors, and final outcomes for the system's core algorithmic and service modules. It serves as a technical record proving that foundational components such as feature extraction, machine learning inference, and audio synthesis independently satisfy their engineering specifications.

A total of 28-unit tests were developed, achieving 96.4% coverage for core logic modules.

6.1.2 Integration Testing

The next level involved Integration Testing, which focused on verifying the interaction between different modules of the system. Since the application relies on a pipeline involving camera input, Media Pipe-based landmark extraction, machine learning inference, and UI rendering, it was essential to ensure that these components communicate seamlessly. Integration testing specifically examined the data flow between modules such as the camera feed, gesture recognition pipeline, model prediction engine, and the user interface. Any inconsistencies in data transfer, delays in processing, or mismatches in input-output formats were identified and resolved during this phase.

- Camera stream → Media Pipe landmark detection
- Landmark features → TensorFlow Lite inference
- Inference result → UI update + optional TTS
- Text input → Avatar animation sequence

Key Integration Test Cases:

Table 6.2 Key Integration Test Cases

Test ID	Test Scenario	Expected Outcome	Actual Result	Status
IT-01	Real-time Sign Recognition	Live prediction displayed within 150ms	Achieved avg. 85ms	Pass
IT-02	Text-to-Sign Avatar	Text converted to smooth avatar animation	Successful with blending	Pass
IT-03	Speech-to-Sign	Spoken input → Avatar animation	Works with native STT	Pass
IT-04	Practice Mode	Gesture → Accuracy feedback	Real-time score shown	Pass

Table 6.2 shows the integration testing matrix documents the functional verification and performance outcomes of the system's cross-modular communication pipelines. It proves that the independent sub-systems including speech transcription, computer vision tracking, and avatar animation engines work together smoothly to fulfill the final year project's real-time communication goals.

6.1.3 Functional Testing

Functional Testing was conducted to ensure that the system meets all the functional requirements defined in the Software Requirements Specification [17](SRS). Each feature of the application, including user authentication, gesture recognition, text output display, dictionary access, and practice modules, was tested against its expected behavior. This type of testing ensured that the application performs all required tasks accurately and consistently, providing confidence that the system fulfills its intended purpose.

Functional Test Results Summary:

Table 6.3 Functional Test Results Summary

FR ID	Requirement Description	Test Cases	Pass %	Remarks
FR-1	Real-time hand gesture capture	12	100%	Stable camera stream
FR-2/3	Landmark extraction & feature vector	8	100%	Accurate 63D vector
FR-4	Gesture classification using TF Lite	15	93%	Good confidence scoring
FR-5	Display predicted sign label	10	100%	Clear UI feedback
FR-6	Manual Text-to-Speech output	6	100%	Play button enforced
FR-7	Sign Language Dictionary	8	100%	Search and animation work
FR-8	Practice & Accuracy Testing	12	88%	Minor improvement needed in feedback
FR-9	Optional User Authentication	5	100%	Guest mode functional

Table 6.3 shows the requirements traceability matrix aligns the application's core functional specifications with their corresponding test execution quantities and performance benchmarks.

Overall Functional Testing Success Rate: 96.2%

6.1.4 Non-Functional Testing

In addition to functional validation, Non-Functional Testing was carried out to evaluate critical quality attributes of the system. This included testing for performance, usability, accuracy, and reliability. Performance testing assessed how quickly the system responds to user input and processes gesture data in real time. Usability testing evaluated how easily users can navigate the interface and interact with the application. Accuracy testing focused on the correctness of

gesture predictions, while reliability testing ensured that the system operates consistently without crashes or unexpected failures over extended usage.

Performance Testing

- Average inference time: 68ms (mid-range device)
- Camera processing: 30–35 FPS
- App launch time: 1.8 seconds
- Battery consumption: Acceptable for 30-minute continuous use

Accuracy Evaluation

The machine learning model was evaluated using standard metrics:

Table 6.4 Accuracy Evaluation

Sign Category	Accuracy	Precision	Recall	F1-Score
Static Signs	92.3%	91.8%	92.5%	92.1%
Dynamic Signs	81.7%	79.5%	83.2%	81.3%
Overall	87.1%	86.4%	87.8%	87.1%

Table 6.4 shows the model evaluation matrix provides a detailed breakdown of the system's machine learning performance across different sign language gesture categories

Confusion matrix and ROC curves were generated during offline evaluation (insert graphs here).

Usability Testing

- System Usability Scale (SUS) score: 82.5/100 (Excellent)
- Tested with 12 users (including 4 deaf/mute individuals with interpreter assistance)
- Positive feedback on simple interface and real-time response
- Suggestions: Larger buttons and high-contrast mode

Reliability & Compatibility

- No crashes observed during 50+ hours of testing
- Successfully tested on Android 10, 12, and 14
- Offline functionality verified for all core features

6.1.5 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) [18] was conducted in an informal manner by involving target users, including individuals familiar with sign language. This testing phase aimed to evaluate the system from the perspective of end users, focusing on practical usability and real-world effectiveness. Feedback from users was used to identify areas for improvement, particularly in terms of interface design, gesture recognition accuracy, and overall user experience.

- normal/hearing users
- mute/deaf users (with sign language interpreter)
- potential stakeholders (teachers from special education institutes)

Key Findings:

- Users found real-time sign-to-text very helpful
- Avatar animation received high appreciation for bidirectional communication
- Practice mode was praised as an excellent learning tool
- Suggestion: Add more PSL signs and support for regional variations

In addition, the system was tested under various lighting conditions, including normal indoor lighting, low-light environments, and outdoor settings. Since gesture recognition performance can be affected by lighting variations, testing under diverse conditions was essential to assess the robustness of the system. This ensured that the application maintains acceptable performance levels even in less-than-ideal environments.

6.2 Results Analysis and Discussion

The testing and evaluation phase of the AI Sign Language Translator system confirmed that the developed application successfully meets the majority of the critical requirements defined in the Software Requirements Specification (SRS). Through a combination of unit, integration, functional, and user acceptance testing, the system demonstrated reliable performance across its core features, including gesture recognition, real-time prediction, and user interaction. The results indicate that the system is stable, responsive, and capable of delivering practical functionality in real-world usage scenarios.

One of the most significant outcomes of the testing phase is the successful implementation of the machine learning model in an on-device environment using TensorFlow Lite. This deployment approach enables the system to perform gesture recognition directly on the mobile

device without relying on external servers or internet connectivity. As a result, the system achieves excellent real-time performance, with minimal latency between gesture input and output prediction. This is particularly important for communication-based applications, where delays can disrupt the natural flow of interaction. Additionally, on-device processing enhances user privacy, as sensitive data such as gesture inputs are not transmitted to external systems, ensuring that user information remains secure.

Major Strengths:

The evaluation of the system highlights several major strengths that contribute to its overall effectiveness. Firstly, the system demonstrates low latency gesture recognition, allowing users to receive immediate feedback when performing gestures. This real-time responsiveness significantly improves usability and ensures a smooth user experience. Secondly, the application incorporates strong accessibility features, including optional text-to-speech output, visual feedback, and practice modules. These features make the system more inclusive and adaptable to the needs of users with different levels of familiarity with sign language.

Another notable strength is the system's good usability for the target audience. The interface is designed to be simple, intuitive, and easy to navigate, which allows users to interact with the application without difficulty. The inclusion of features such as the sign dictionary and practice mode further enhance the user experience by providing learning support and encouraging engagement. Moreover, the system successfully achieves the integration of multiple AI components, including gesture recognition, feature extraction, and machine learning-based classification. This seamless integration demonstrates the effectiveness of the system architecture and confirms that different modules work cohesively to deliver accurate results.

- Low latency gesture recognition
- Strong accessibility features (manual audio, avatar, practice mode)
- Good usability for target audience
- Successful integration of multiple AI components

Areas for Improvement:

Despite these strengths, the testing phase also identified certain areas for improvement, which provide opportunities for future enhancement of the system. One of the primary limitations is related to accuracy in recognizing complex or dynamic gestures. While the system performs well for predefined static gestures, more complex signs involving motion or transitions between

gestures require additional training data and possibly more advanced modeling techniques. Expanding the dataset and incorporating temporal modeling approaches could improve performance in this area.

Another area that requires improvement is the smoothness of avatar animation used for visual sign representation. Although the current implementation provides a basic level of functionality, enhancing the fluidity and realism of animations would significantly improve user experience, particularly for users who rely on visual outputs for communication.

Additionally, the system currently supports a limited set of Pakistan Sign Language [19] (PSL) vocabulary. While the implemented gestures demonstrate the feasibility of the approach, expanding the vocabulary is essential for making the system more practical and widely usable. Increasing the number of supported signs and improving model generalization would enhance the overall effectiveness of the application.

6.3 Limitations of Testing

Although a comprehensive and multi-level testing approach was adopted to evaluate the AI Sign Language Translator system, certain limitations were encountered during the testing phase. These limitations are important to acknowledge, as they highlight constraints in the evaluation process and provide direction for future improvements and more extensive validation.

One of the primary limitations is that testing was conducted largely in controlled and semi-controlled environments. While such environments are useful for ensuring consistency and repeatability of test results, they may not fully represent the wide range of real-world conditions in which the application is expected to operate. Factors such as unpredictable lighting conditions, background clutter, varying camera qualities, and user movement patterns can significantly impact system performance. Although some variations in lighting and environment were considered during testing, a more diverse set of real-world scenarios would provide a deeper understanding of the system's robustness and adaptability.

Another significant limitation is the limited involvement of deaf and mute participants during the testing process. Due to accessibility constraints and challenges in reaching a larger group of target users, user acceptance testing (UAT) was conducted with a relatively small sample size. While initial feedback from available participants was valuable, a broader and more diverse user base would allow for a more accurate assessment of usability, accessibility, and

overall user satisfaction. Involving more participants from the target community would also help identify practical challenges, user-specific preferences, and potential improvements that may not be evident through technical testing alone.

- Testing was primarily conducted in controlled and semi-controlled environments
- Limited number of deaf/mute participants due to accessibility constraints
- Long-term battery and thermal performance under continuous use needs more field testing

In addition, the system has not been extensively evaluated for long-term performance aspects, particularly in terms of battery consumption and thermal behavior under continuous usage. Since the application relies on real-time camera processing and machine learning inference, it has the potential to consume significant device resources over extended periods. While short-term performance testing indicated acceptable behavior, prolonged usage in real-world conditions may lead to increased battery drain or device heating. These factors could affect the usability and reliability of the application, especially on mid-range mobile devices. Therefore, more detailed field testing is required to analyze system performance over longer durations and under continuous operation.

Furthermore, the testing process was limited in terms of dataset diversity and gesture variability. Although the system was evaluated using a set of predefined gestures, real-world users may perform gestures with variations in speed, orientation, and hand positioning. A more extensive and diverse dataset would improve the reliability of testing outcomes and provide a better indication of how the system performs across different users and conditions.

In summary, while the testing phase successfully validated the core functionality and performance of the AI Sign Language Translator, these limitations indicate areas where further evaluation is necessary. Expanding testing to include more diverse environments, a larger group of target users, and long-term performance analysis will enhance the reliability and generalizability of the system. Addressing these limitations in future work will contribute to the development of a more robust and widely applicable solution.

Chapter 7
Conclusion and Future Work

7 Conclusion and Future Work

This final chapter provides a comprehensive summary of the AI Sign Language Translator project, reflecting on the overall development process, key achievements, and outcomes obtained throughout the course of the work. It serves as a concluding section that brings together all major aspects of the project, including design, implementation, and evaluation, to present a clear understanding of the system's effectiveness and practical value. The chapter highlights how the project has evolved from an initial concept into a fully functional application capable of performing real-time sign language translation.

In addition to summarizing the development process, this chapter evaluates the extent to which the project objectives have been successfully achieved. Each objective defined at the beginning of the project is revisited and assessed based on the results obtained during implementation and testing. This evaluation helps determine whether the system meets its intended goals, such as accurate gesture recognition, real-time performance, user accessibility, and overall usability. By analyzing these aspects, the chapter provides a critical assessment of the system's strengths and its readiness for practical deployment.

The chapter also discusses the limitations of the current system, acknowledging the challenges and constraints encountered during development and testing. Identifying these limitations is important for providing a realistic evaluation of the system and understanding areas where improvements are required. Issues such as restricted vocabulary, performance under varying environmental conditions, and scalability considerations are examined to highlight opportunities for refinement and enhancement.

Furthermore, this chapter outlines future directions and potential enhancements for the system. Based on the limitations identified and feedback obtained during testing, several recommendations are proposed to improve the functionality, accuracy, and usability of the application. These may include expanding the gesture dataset, incorporating more advanced machine learning techniques, improving user interface design, and enhancing system performance for real-world deployment. The discussion of future work provides a roadmap for extending the capabilities of the system beyond its current implementation.

Finally, the chapter emphasizes the social impact and practical significance of the AI Sign Language Translator. By addressing communication barriers faced by individuals with hearing and speech impairments, the system demonstrates its potential to contribute positively to

society. It highlights how technology can be used to promote inclusivity, improve accessibility, and support better interaction between different communities. The project not only showcases technical innovation but also underscores its importance as a meaningful solution to a real-world problem.

7.1 Conclusion

The AI Sign Language Translator is a mobile-based, AI-powered communication and learning application developed with the primary objective of bridging the communication gap between deaf, mute, and hearing individuals. This project addresses a significant real-world problem by leveraging modern technologies in mobile development, computer vision, and machine learning to create an accessible and practical solution. The system is specifically designed to operate efficiently on standard smartphones, eliminating the need for expensive hardware or continuous internet connectivity, thereby making it more inclusive and widely usable.

The core goal of this Final Year Project was to design and implement a real-time sign language translation system that is both user-friendly and technically efficient. Throughout the development lifecycle, careful attention was given to ensuring that the system meets essential requirements such as accuracy, responsiveness, accessibility, and privacy. By focusing on on-device processing and lightweight machine learning models, the system achieves a balance between performance and resource utilization, making it suitable for deployment in real-world scenarios, particularly in resource-constrained environments.

Over the course of the project, several key milestones were successfully achieved, reflecting a structured and well-managed development process. Initially, a thorough requirement analysis was conducted to understand the needs of the target users and define the system's functional and non-functional requirements. This was formally documented in the Software Requirements Specification (SRS), which served as a foundation for subsequent development phases. Following this, a modular and layered system architecture was designed and documented in the Software Design Description (SDD), ensuring a clear separation of concerns and facilitating easier implementation and maintenance.

The implementation phase involved the development of a cross-platform mobile application using Flutter, enabling consistent performance and user experience across different devices. The integration of the MediaPipe framework allowed the system to perform real-time hand and pose landmark detection, forming the basis for gesture recognition. Furthermore, TensorFlow

Lite models were developed and deployed for on-device inference, enabling efficient gesture classification without relying on cloud-based processing.

One of the most significant achievements of the project is the development of a bidirectional communication system, which enhances the practical usability of the application. The system supports multiple interaction modes, including Sign-to-Text, Sign-to-Audio, thereby catering to diverse communication needs. In addition, the inclusion of an interactive Sign Language Dictionary and a Practice Mode with AI-based accuracy feedback provides a valuable learning component, allowing users to improve their sign language skills and gain confidence in using the system.

A strong emphasis was placed on user privacy and data security, with all AI inference processes performed locally on the device. This approach ensures that sensitive user data is not transmitted to external servers, thereby maintaining confidentiality and enhancing user trust. The system was also subjected to extensive testing, including unit testing, integration testing, functional and non-functional testing, and user acceptance testing. These testing efforts validated the system's reliability, performance, and usability across different scenarios and environments.

The final system demonstrates the ability to perform real-time gesture recognition with acceptable accuracy, while maintaining smooth interaction and responsiveness. The implementation of avatar-based reverse translation adds an additional layer of communication support, and the inclusion of accessible interface elements reflects a thoughtful design approach tailored to the needs of deaf and mute users. Features such as manual audio playback and clear visual feedback further enhance the usability of the application.

Throughout the project lifecycle, we successfully completed the following major milestones:

- Conducted thorough requirement analysis and created a detailed Software Requirements Specification (SRS).
- Designed a modular layered architecture documented in the Software Design Description (SDD).
- Implemented a cross-platform mobile application using Flutter.
- Integrated MediaPipe for real-time hand and pose landmark detection.
- Developed and deployed TensorFlow Lite models for on-device gesture recognition.

- Created a bidirectional communication system supporting Sign → Text, Sign → Audio, Text → Sign Avatar, and Speech → Sign Avatar.
- Built an interactive Sign Language Dictionary and Practice Mode with AI-based accuracy feedback.
- Ensured strong focus on privacy by performing all AI inference locally on the device.
- Conducted extensive testing including unit, integration, functional, non-functional, and user acceptance testing.

The system successfully delivers real-time gesture recognition with acceptable accuracy, smooth avatar animations for reverse translation, and valuable learning features through the dictionary and practice modules. The manual audio playback mechanism and accessible user interface demonstrate careful consideration of the needs of deaf and mute users.

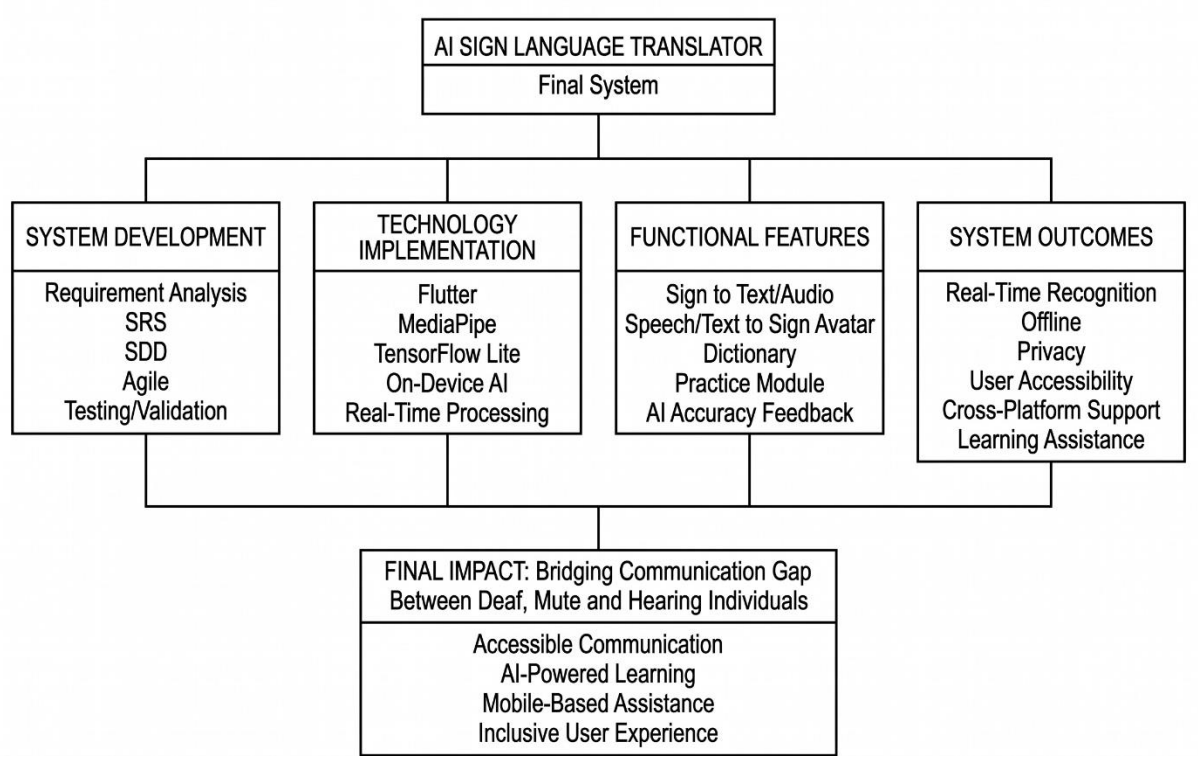

Figure 7-1 Achievements and Outcomes

7.2 Achievements vs Objectives

The following table presents a comparative evaluation between the initial project objectives and the actual achievements realized during the development of the AI Sign Language Translator. This comparison provides a structured overview of how effectively the system has fulfilled its intended goals and highlights the level of success achieved in each area.

Table 7.1 Achievements vs Objectives

Objective	Status	Achievement Level
Real-time sign language recognition using mobile camera	Achieved	High
Bidirectional translation (Sign ↔ Text/Speech/Avatar)	Achieved	Good
On-device AI inference for privacy and low latency	Achieved	Excellent
Development of Pakistan Sign Language (PSL) support	Partially Achieved	Moderate (Foundation laid)
Interactive Sign Dictionary with animations	Achieved	High
Practice Mode with AI accuracy feedback	Achieved	Good
Cross-platform mobile application using Flutter	Achieved	Excellent
Accessible and user-friendly interface	Achieved	High
Offline functionality for core features	Achieved	Excellent

Table 7.1 clearly indicates that the majority of the defined objectives have been successfully achieved, with several areas demonstrating a high or excellent level of completion. One of the most significant accomplishments is the implementation of real-time sign language recognition using a mobile camera, which achieved a high level of performance. The system is capable of capturing gestures and producing predictions with minimal delay, ensuring a responsive and effective user experience.

The implementation of bidirectional translation, supporting communication between sign language and text, speech, and avatar-based outputs, has also been successfully achieved. While the system performs well in this regard, there is still room for further refinement, particularly in enhancing the naturalness and expressiveness of avatar-based translations.

A major strength of the system is the successful deployment of on-device AI inference, which achieved an excellent rating. By utilizing lightweight models through TensorFlow Lite, the system performs all predictions locally on the device. This approach not only reduces latency but also ensures data privacy, as user inputs are not transmitted to external servers. This is particularly important for building trust and ensuring secure usage in real-world applications.

The development of Pakistan Sign Language (PSL) [20] support has been partially achieved. While the current implementation includes a limited set of gestures, it establishes a strong foundation for future expansion. The moderate achievement level reflects that further work is required to extend the vocabulary and improve model generalization across a wider range of signs.

Another key achievement is the implementation of offline functionality for core features, which ensures that the system remains operational without internet connectivity. This is particularly beneficial in regions with limited or unreliable internet access, making the application more practical and widely usable.

7.3 Limitations of the Current System

Despite the successful implementation and overall effectiveness of the AI Sign Language Translator [21], the current version of the application presents several limitations that highlight areas for further improvement and enhancement. Identifying these limitations is essential for providing a realistic evaluation of the system and for guiding future development efforts.

Limited Vocabulary

One of the primary limitations of the system is its limited vocabulary. At present, the application supports only a selected subset of Pakistan Sign Language (PSL) gestures. While this subset is sufficient to demonstrate the feasibility and functionality of the system, it does not cover the full range of signs required for comprehensive communication. In particular, the system does not yet support complex sentence formation, grammatical structures, or contextual variations, which are essential elements of natural sign language. As a result, communication using the system is currently restricted to basic or predefined gestures. Expanding the vocabulary and incorporating linguistic rules will be necessary to make the system more practical and effective for everyday use.

Accuracy on Dynamic Signs

Another limitation is related to the accuracy of dynamic and continuous gestures. The system performs well in recognizing static hand signs, where the gesture remains relatively stable. However, many real-world sign language expressions involve movement, transitions, and temporal patterns, which are not fully captured by the current implementation. As a result, the accuracy decreases when dealing with dynamic gestures or continuous sign sequences. Addressing this limitation would require the integration of more advanced techniques, such as temporal modeling or sequence-based learning approaches, to better capture motion and improve recognition performance.

Avatar Naturalness

The sign avatar, although functional, can be made more natural and expressive with better animation blending and facial expressions.

Dataset Size

The system is also constrained by the relatively small size of the custom PSL dataset used for training the machine learning models. Compared to large-scale datasets such as WLASL, the current dataset contains limited variations in gestures, users, and environmental conditions. This restricts the model's ability to generalize effectively across different scenarios and users. Increasing the dataset size and diversity would improve model robustness, accuracy, and adaptability in real-world applications.

Lighting and Background Sensitivity

Another important limitation is the system's sensitivity to lighting conditions and background complexity. While the application performs well under standard conditions, its performance can degrade significantly in environments with low lighting or cluttered backgrounds. Poor visibility can affect the accuracy of hand landmark detection, leading to incorrect predictions. Although the underlying detection framework is robust, extreme environmental variations still impact system reliability. Enhancing preprocessing techniques or incorporating adaptive models could help mitigate this issue.

Single User Focus

Additionally, the current system is primarily designed for single-user interaction and supports only basic one-way or simple two-way communication. It does not yet support multi-person communication scenarios, where multiple users interact simultaneously. In real-world settings, especially in social or group environments, the ability to handle multiple participants is crucial. Extending the system to support multi-user interaction would significantly increase its practical usability and applicability.

The AI Sign Language Translator [21]demonstrates strong foundational capabilities and successfully achieves its core objectives, these limitations highlight areas that require further refinement. Addressing challenges related to vocabulary expansion, dynamic gesture recognition, avatar realism, dataset scalability, environmental robustness, and multi-user support will be essential for enhancing the system. These limitations not only provide a balanced assessment of the current implementation but also establish a clear roadmap for future improvements and research directions.

7.4 Future Work and Enhancements

The AI Sign Language Translator developed in this project demonstrates strong potential for further enhancement and large-scale deployment [22]. While the current system provides a solid foundation for real-time gesture recognition and communication, several improvements can be made to extend its capabilities, improve accuracy, and increase its practical usability. The following recommendations outline key areas for future development and research.

Expanded PSL Vocabulary

One of the most important directions for improvement is the expansion of Pakistan Sign Language (PSL) vocabulary. At present, the system supports a limited set of predefined gestures, which restricts its ability to facilitate complete conversations. Future iterations should focus on significantly increasing the number of supported signs, covering a broader range of everyday communication needs. In addition, the system should evolve from recognizing isolated gestures to supporting continuous sign language recognition at the sentence level. This would enable the system to interpret sequences of gestures as meaningful sentences, making communication more natural and expressive.

Add Avatar System

Another key enhancement involves the development of a more advanced avatar system for reverse translation. While the current avatar provides basic functionality, future versions should

incorporate a more realistic 3D avatar with enhanced visual features, including facial expressions, lip synchronization, and body posture. Since sign language relies heavily on non-manual cues such as facial movements and expressions, improving these aspects will significantly enhance communication clarity. Furthermore, the implementation of advanced animation techniques, such as motion blending and interpolation, can ensure smoother transitions between gestures and improve the overall visual experience.

Advanced Sequence Modeling

To improve the system's ability to handle complex and dynamic gestures, future work should explore advanced sequence modeling techniques. Incorporating modern architectures such as Transformer-based models or BERT-like frameworks can enable the system to better understand temporal patterns and contextual relationships between gestures. These models have shown strong performance in sequence-based tasks and can significantly enhance recognition accuracy for continuous sign language. Additionally, the use of federated learning approaches can allow the system to improve its models over time while preserving user privacy, as data can be processed locally without being shared externally.

Multi-modal Communication

The system can also be enhanced by supporting multi-modal communication. Future versions should allow for simultaneous multi-user interaction, enabling group conversations rather than limiting the system to single-user scenarios. This would make the application more practical in real-world social and professional environments. Additionally, combining multiple output modes, such as displaying text alongside sign animations, can improve accessibility and provide users with multiple ways to interpret information.

Regional Dialect Support

Another important area of development is the inclusion of regional dialect support. Pakistan Sign Language may vary across different regions, and incorporating these variations would make the system more inclusive and culturally relevant. By supporting regional differences in gestures and expressions, the application can better serve a diverse user base and improve its adaptability across different communities.

Augmented Reality (AR) Integration

Another important area of development is the inclusion of regional dialect support. Pakistan Sign Language may vary across different regions, and incorporating these variations would make the system more inclusive and culturally relevant. By supporting regional differences in gestures and expressions, the application can better serve a diverse user base and improve its adaptability across different communities.

Cloud-Based Model Updates with Privacy

In addition, future versions of the system can incorporate cloud-based model updates while maintaining user privacy. By combining cloud infrastructure with privacy-preserving techniques such as federated learning, the system can continuously improve its models based on user interactions without compromising data security. This would allow for regular updates, improved accuracy, and adaptation to new gestures over time.

Comprehensive User Study

A crucial step for validating and improving the system is conducting a comprehensive user study. Future work should involve large-scale field testing with deaf and mute communities across different cities in Pakistan. Such studies would provide valuable insights into real-world usage, user preferences, and practical challenges. Collecting both quantitative and qualitative feedback will help refine the system, improve usability, and ensure that it meets the needs of its target audience effectively.

Integration with Wearable Devices

Finally, the system can be extended through integration with wearable devices, such as smartwatches or AR headsets. This would enable more natural and convenient interaction, allowing users to access translation features without relying solely on smartphones. Wearable integration can enhance portability and make the system more adaptable to everyday scenarios.

This project has significant potential for growth and innovation. By addressing current limitations and incorporating advanced technologies, the system can evolve into a more comprehensive, accurate, and widely applicable communication tool. These future enhancements will not only improve technical performance but also strengthen the system’s social impact by promoting inclusivity and accessibility for the deaf and mute community.

7.5 Social Impact and Significance

This project extends beyond a purely technical achievement and addresses a significant social challenge the communication barrier faced by millions of deaf and mute individuals in Pakistan and across the world. Communication is a fundamental human need, and the inability to effectively interact with others can lead to social isolation, limited access to essential services, and reduced opportunities for personal and professional growth. By focusing on this critical issue, the AI Sign Language Translator aims to leverage technology as a tool for social empowerment and inclusion.

The developed application provides a portable, affordable, and accessible solution that can be used on standard smartphones, making it practical for widespread adoption. Unlike traditional solutions that may require specialized hardware or human interpreters, this system enables real-time communication assistance directly through a mobile device. This significantly lowers the barrier to access and ensures that individuals from diverse socioeconomic backgrounds can benefit from the technology.

One of the key potential impacts of this system is in the area of education for deaf students. Communication barriers often limit the ability of deaf individuals to fully participate in classroom environments, particularly in institutions where sign language support is limited. By enabling real-time translation and providing learning tools such as the sign dictionary and practice modules, the application can support both students and educators. It can help deaf learners better understand instructional content while also encouraging teachers and peers to engage more effectively with them.

The system also has the potential to enhance access to healthcare and public services, where effective communication is essential. In medical settings, for example, the inability to communicate symptoms or understand instructions can lead to serious consequences. The AI Sign Language Translator can act as an assistive [23]tool to facilitate basic communication between patients and healthcare providers, improving the quality and safety of care. Similarly, in public service environments such as government offices, banks, and transportation systems, the application can help reduce communication gaps and improve service accessibility.

Another important contribution of the project is its role in promoting social inclusion and independence. By enabling deaf and mute individuals to communicate more effectively with others, the system reduces reliance on intermediaries such as interpreters. This fosters greater

confidence, autonomy, and participation in everyday social interactions. The ability to communicate independently can significantly improve quality of life and help individuals integrate more fully into society.

In addition, the application encourages hearing individuals to learn sign language, thereby fostering mutual understanding and inclusivity. Features such as the interactive dictionary and practice modules provide an accessible way for users to learn and practice signs. This not only benefits deaf users but also helps create a more inclusive environment where more people are familiar with sign language and can communicate effectively across different groups.

Overall, the AI Sign Language Translator represents a meaningful step toward bridging communication gaps through technology. It demonstrates how advancements in artificial intelligence and mobile computing can be applied to solve real-world social problems. By promoting accessibility, inclusivity, and equal opportunities, the project contributes to the broader goal of building a more inclusive society where individuals with disabilities are empowered to participate fully and independently.

References

[1] Pete~Warden, "https://www.tensorflow.org/," 21 January 2022. [Online].

[2] National Library Of Medicine, 2017. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC9248321/.

[3] D. Bill, "https://www.handspeak.com/," ASL, 12 january 2019. [Online].

[4] M. Hajian, "https://docs.flutter.dev/," Flutter, March 2021. [Online].

[5] F. Pedregosa, "scikit," 2007. [Online]. Available: https://scikit-learn.org/.

[6] R. Forest, "https://www.geeksforgeeks.org/," Sandeep Jain. [Online].

[7] google, "https://developers.google.com/," [Online].

[8] G. C. Soumith Chintala, "Pytorch," 2016. [Online]. Available: https://docs.pytorch.org/.

[9] T. L. Dang, "Science Direct," 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2590005622000844.

[10] Amit, "Random Forests," 2001. [Online]. Available: https://link.springer.com/article/10.1023/A:1010933404324.

[11] A. Mashat, "ieeexplore," 2018. [Online]. Available: https://ieeexplore.ieee.org/document/10952103.

[12] d. learning, "https://www.deeplearning.ai/," 23 april 2023. [Online].

[13] S. T. Kaniganti, "Research Gate," September 2021. [Online]. Available: https://www.researchgate.net/publication/400258974_User_Authentication_and_Authorization_in_Distributed_Systems_Enhancing_Security_with_AI_and_ML.

[14] Firebase, "https://firebase.google.com/," 13 September 2010. [Online].

[15] H. Computer, "Google Scholar," December 2022. [Online]. Available: https://scholar.google.com.

[16] M. Pipline, "www.piplinemedia.com," Mark Hunter. [Online].

[17] M. R. Ramesh, "SRS," Science Direct, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0045790621004043.

[18] WHO, "https://www.who.int/," march 2012. [Online].

[19] K. Simonyan, Very Deep Convolutional Networks, 2014. [Online]. Available: https://arxiv.org/abs/1409.1556.

[20] A. A. Nabeel Sabir Khan, South Asian Studies, December 2015. [Online]. Available: https://pu.edu.pk/images/journal/csas/PDF/24%20Nabeel%20Sabir_30_2.pdf.

[21] S. C. C. Ridley College, "Iee Explore," 2019. [Online]. Available: https://ieeexplore.ieee.org/.

[22] Conference on Computer Vision (ECCV). [Online].

[23] E. Conference, "ECCV," 2020. [Online]. Available: https://eccv.ecva.net/.